

MICROSAR CAN Driver

Technical Reference

Infineon

Tricore

MultiCAN ®

Version 1.28.00

Authors	Holger Birke, Rüdiger Naas, Han Zhao
Status	Released

Document Information

1.1 History

Author	Date	Version	Remarks
Holger Birke	2006-06-21	1.00.00	Initial version
Holger Birke	2006-06-28	1.01.00	Review modifications
Holger Birke	2006-08-24	1.02.00	Update to ASR 2.1 and new template
Holger Birke	2006-09-13	1.03.00	Update Core
Holger Birke	2007-11-21	1.04.00	Update Core
Holger Birke	2008-01-21	1.05.00	Add new derivative 1797
Holger Birke	2008-03-20	1.06.00	Update to Template 1.11 Update to ASR3
Holger Birke	2008-05-08	1.07.00	Some minor changes due to review
Holger Birke	2008-09-02	1.08.00	Update to Core 3.3 (Configuration add-ons)
Holger Birke	2008-09-26	1.09.00	GUI names in GENy changed to AutoSar naming
Holger Birke	2008-09-26	1.10.00	Add derivative 1767, 1xxx update GENy GUI + update to new template
Holger Birke	2009-05-19	1.11.00	Improve sleep description (limitation for API while sleep)
Holger Birke	2009-06-08	1.12.00	Update to template 1.18 (Core 3.8)
Honert Heike	2009-11-02	1.12.01	Correct bookmarks
Holger Birke	2010-02-15	1.13.00	Update to template 1.19.00 (Core 3.11)
Holger Birke	2010-05-11	1.13.01	Some minor fixes
Holger Birke	2011-07-04	1.13.02	Add multiple ECU configuration support, Add List of Supported Derivatives, Add support for fractional divider
Holger Birke	2011-11-16	1.14.00	Update to Core 4.01
Holger Birke	2011-12-02	1.14.01	Add Appl_Lock / UnlockInit() / Appl_EnableInt ESCAN00051922
Holger Birke	2012-02-03	1.15.00	ESCAN00056335: Support Protected Mode
Holger Birke	2012-05-22	1.16.00	ESCAN00057652: Wrong API description for Generic Pretransmit Add Derivatives TC1728, TC2xx
Holger Birke	2012-11-26	1.17.00	Update to Core 4.04
Holger Birke	2013-07-18	1.18.00	Add Cfg5 feature list
Holger Birke	2013-07-29	1.18.01	Improve Multiple Driver description
Holger Birke	2013-10-24	1.19.00	Update to Core 4.06
Holger Birke	2014-02-27	1.20.00	Add Derivatives TC264, TC274, TC297
Rüdiger Naas	2014-04-02	1.21.00	Add compiler Diabdata

Peter Herrmann	2014-04-14	1.22.00	Add Derivatives TC233, TC234, TC223. Extend description of „Multiple Driver“ feature.
Holger Birke	2014-09-26	1.22.01	Add CAN_PROTECTED_AREA_SRC
Rüdiger Naas	2015-03-11	1.23.00	Update to Core 4.10
Rüdiger Naas	2015-05-19	1.23.01	ESCAN00081377 Harmonization of EndInit Protection
Rüdiger Naas	2015-11-18	1.24.00	Rx HW FIFO buffer added Add hints and restrictions for ‚Multiple CAN Cells‘
Rüdiger Naas	2015-11-18	1.25.00	Update to Core 4.12
Rüdiger Naas	2016-04-15	1.26.00	- ESCAN00089530 Add implementation hint regarding "nested CAN interrupts" in conjunction with GNU compiler - Functional description: „Common CAN“ removed - Extended RAM check
Rüdiger Naas	2016-07-20	1.27.00	- ESCAN00090355: OS Cat2 IR prio configuration - Add InterruptController by OS handling - FIFO behavior description
Han Zhao	2017-12-15	1.28.00	- Add support of Silent Mode - Add support of Selective Multiplexed Tx

Reference Documents

No.	Title	Version
[1]	AUTOSAR_SWS_CAN_DRIVER.pdf	2.4.6 + 3.0.0 + 4.0.0
[2]	AUTOSAR_BasicSoftwareModules.pdf	V1.0.0
[3]	AUTOSAR_SWS BSW Scheduler	V1.1.0
[4]	AUTOSAR_SWS_CAN_Interface.pdf	3.2.7 + 4.0.0 + 5.0.0
[5]	AN-ISC-8-1118 MICROSAR BSW Compatibility Check	V1.0.0

Table 2-1 Referenced documents

1.2 Scope of the Document

This document describes the functionality, API and configuration of the MICROSAR CAN driver as specified in [1]. The CAN driver is a hardware abstraction layer with a standardized interface to the CAN Interface layer.

**Caution**

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1.1	History	2
1.2	Scope of the Document.....	3
2	Hardware Overview	8
2.1	Supported Derivatives.....	9
3	Introduction.....	10
3.1	Architecture Overview	10
4	Functional Description.....	12
4.1	Features	12
4.2	Initialization	15
4.3	Communication	15
4.4	States / Modes	18
4.5	Re-Initialization	19
4.6	CAN Interrupt Locking.....	19
4.7	Main Functions	19
4.8	Error Handling.....	19
4.9	CAN RAM Check	22
4.10	Extended RAM Check.....	23
4.11	Hardware Receive FIFO buffer	23
4.12	Hardware Specific.....	24
5	Integration.....	25
5.1	Scope of Delivery.....	25
5.2	Include Structure.....	26
5.3	Critical Sections	26
5.4	Compiler Abstraction and Memory Mapping.....	28
5.5	Partial Network	29
5.6	Hardware Specific Hints.....	29
5.7	Multiple Can Driver	30
5.8	EndInit Protection.....	31
5.9	Protected Register Access	32
5.10	Nested CAN interrupts in conjunction with GNU compiler	32
6	API Description.....	33
6.1	Interrupt Service Routines provided by CAN	33
6.2	Services provided by CAN	34
6.3	Services used by CAN	61

7	Configuration.....	63
7.1	Pre-Compile Parameters.....	63
7.2	Link-Time Parameters.....	64
7.3	Post-Build Parameters.....	65
7.4	Configuration with GENy.....	65
7.5	Configuration with da DaVinci Configurator.....	80
8	AUTOSAR Standard Compliance.....	81
8.1	Limitations / Restrictions.....	81
8.2	Vector Extensions.....	81
9	Glossary and Abbreviations.....	82
9.1	Glossary.....	82
9.2	Abbreviations.....	82
10	Contact.....	83

Illustrations

Figure 3-1	AUTOSAR architecture.....	11
Figure 3-2	Interfaces to adjacent modules of the CAN.....	11
Figure 5-1	Include Structure (AUTOSAR).....	26
Figure 6-1	Select OS Type.....	33
Figure 7-1	Platform settings.....	65
Figure 7-2	Individual polling.....	74
Figure 7-3	Common CAN channel setting.....	75
Figure 7-4	Controller Settings.....	75
Figure 7-5	Init Structure Dialog.....	77
Figure 7-6	Setup Filter Dialog.....	77
Figure 7-7	Baud Rate Dialog.....	79

Tables

Table 2-1	Referenced documents.....	3
Table 2-1	Supported Hardware Overview.....	8
Table 2-2	Supported derivatives.....	9
Table 4-1	Supported features.....	15
Table 4-2	Hardware mailbox layout.....	16
Table 4-3	Errors reported to DET.....	20
Table 4-4	API from which the Errors are reported.....	21
Table 4-5	Errors reported to DEM.....	22
Table 4-6	Hardware Loop Check.....	22
Table 5-1	Static files.....	25
Table 5-2	Generated files.....	25
Table 5-3	Critical Section Codes.....	28
Table 5-4	Compiler abstraction and memory mapping.....	29
Table 6-1	Can_InitMemory.....	35
Table 6-2	Can_Init.....	36
Table 6-3	Can_InitController.....	36

Table 6-4	Can_ChangeBaudrate	37
Table 6-5	Can_CheckBaudrate	37
Table 6-6	Can_SetBaudrate	38
Table 6-7	Can_InitStruct.....	39
Table 6-8	Can_GetVersionInfo	39
Table 6-9	Can_GetStatus	40
Table 6-10	Can_SetControllerMode	41
Table 6-11	Can_ResetBusOffStart	41
Table 6-12	Can_ResetBusOffEnd	42
Table 6-13	Can_Write.....	43
Table 6-14	Can_CancelTx.....	43
Table 6-15	Can_CheckWakeup.....	44
Table 6-16	Can_DisableControllerInterrupts.....	44
Table 6-17	Can_EnableControllerInterrupts.....	45
Table 6-18	Can_MainFunction_Write	45
Table 6-19	Can_MainFunction_Read	46
Table 6-20	Can_MainFunction_BusOff.....	46
Table 6-21	Can_MainFunction_Wakeup.....	47
Table 6-22	Can_MainFunction_Mode.....	47
Table 6-23	Can_RamCheckExecute	48
Table 6-24	Can_RamCheckEnableMailbox	49
Table 6-25	Can_RamCheckEnableController	49
Table 6-26	CanInterrupt	50
Table 6-27	Appl_GenericPrecopy	50
Table 6-28	Appl_GenericConfirmation.....	51
Table 6-29	Appl_GenericPreTransmit.....	52
Table 6-30	ApplCanTimerStart	52
Table 6-31	ApplCanTimerLoop.....	53
Table 6-32	ApplCanTimerEnd	54
Table 6-33	ApplCanInterruptDisable.....	54
Table 6-34	ApplCanInterruptRestore	55
Table 6-35	Appl_CanOverrun.....	55
Table 6-36	Appl_CanFullCanOverrun.....	56
Table 6-37	Appl_CanCorruptMailbox.....	56
Table 6-38	Appl_CanRamCheckFailed.....	57
Table 6-39	ApplCanCorruptRegisters	57
Table 6-40	ApplCanRxFiFoFull.....	58
Table 6-41	Appl_UnlockEndinit.....	58
Table 6-42	Appl_LockEndinit.....	59
Table 6-43	Appl_EnableInt	60
Table 6-44	ApplCanWaitSetMCR	60
Table 7-1	Platform Parameter description	66
Table 7-2	Controller Parameter description	76
Table 7-3	Filter Parameter description.....	78
Table 7-4	Baud rate Parameter description	80
Table 9-1	Glossary	82
Table 9-2	Abbreviations.....	82

2 Hardware Overview

The following table summarizes information about the CAN Driver. It gives you detailed information about the derivatives and compilers. As very important information the documentations of the hardware manufacturers are listed. The CAN Driver is based upon these documents in the given version.

Derivative	Compiler	Hardware Manufacturer Document	Version
See the following table "Supported derivatives"	GNU, Tasking, Greenhills, Diabdata	TC1796_tspu_v21_2003-11.pdf TC1796_tssu_v21_2003-11.pdf TC1796_Errata_BC_v1_7.pdf TC1796_AB_Errata_V1_4.pdf TC1796_AC_Errata_V1_2.pdf TC1796_BA_Errata_Sheet_v1_2.pdf TC1796_BB_Errata_Sheet_v1_0.pdf TC1796_BC_Errata_Sheet_v1_1.pdf TC1796_AA_Errata_V10.pdf TC1796_AtoB_DeltaSpec_V12.pdf TC1766_tspu_v21_2004_10.pdf TC1766_tssu_v21_2004_10.pdf TC1766_AC_Errata_v10.pdf TC1766_ED_EES_BA_v10.pdf TC1766_AtoB_DeltaSpec_V11.pdf TC1766_BD_Errata_Sheet_v1_0.pdf TC1797_TS_V1.4-2007-06-22.pdf TC1767_can_V1.5-2007-11.pdf tc1387_ts_can_v1.6.pdf (TC1768) TC1728_TS_V1.1.pdf tc27x_ts_V3.0_OPEN_MARKET.pdf tc23x_tc22x_ts_v2.0.pdf	See PDF File Name

Table 2-1 Supported Hardware Overview

Derivative: This can be a single information or a list of derivatives, the CAN Driver can be used on.

Compiler: List of Compilers the CAN Driver is working with

Hardware Manufacturer Document Name: List of hardware documentation the CAN Driver is based on.

Version: To be able to reference to this hardware documentation its version is very important.

2.1 Supported Derivatives

Supported Derivative	Amount of Channels	Amount of Mailboxes	Compatible Derivatives
TC1xxx (Generic)	selectable	selectable	All derivatives listed below (use settings of Channel and Mailboxes) for CAN cell MultiCAN
TC1796, TC1797, TC1793, TC1791, TC1798	4	128	TC1197
TC1768, TC1782, TC1784, TC1746	3	128	
TC1728, TC1724(N/F)	3	64	
TC1766, TC1767	2	64	TC1166, TC1165
TC2xx (Generic)	selectable	selectable	All derivatives listed below (use settings of Channel and Mailboxes) for CAN cell MultiCAN+ (also SRC base Address and Clock Input has to be configured)
TC2D5, TC264, TC265, TC267 TC274, TC275, TC277	4	256	
TC297, TC298, TC299	4 + 2	256 + 128	
TC212, TC222, TC223	3	128	
TC233, TC234	3 + 3	128 + 128	
TC264 StepB TC265 StepB TC267 StepB	5	256	

Table 2-2 Supported derivatives

3 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CAN as specified in [1].

Since each hardware platform has its own behavior based on the CAN specifications, the main goal of the CAN driver is to give a standardized interface to support communication over the CAN bus for each platform in the same way. The CAN driver works closely together with the higher layer CAN interface.

Supported AUTOSAR Release*:	3 and 4	
Supported Configuration Variants: (Supported AUTOSAR Standard Conform Features)	Pre-Compile Link-Time Post-Build Loadable Post-Build Selectable (MICROSAR Identity Manager)	
Vendor ID:	CAN_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	CAN_MODULE_ID	80 decimal (according to ref. [2])
AR Version:	CAN_AR_RELEASE_MAJOR_VERSION CAN_AR_RELEASE_MINOR_VERSION CAN_AR_RELEASE_REVISION_VERSION	AUTOSAR Release version (BCD coded)
SW Version:	CAN_SW_MAJOR_VERSION CAN_SW_MINOR_VERSION CAN_SW_PATCH_VERSION	MICROSAR Can module version (BCD coded)

* For the precise AUTOSAR Release 3.x and 4.x please see the release specific documentation.

3.1 Architecture Overview

The following figure shows where the CAN is located in the AUTOSAR architecture.

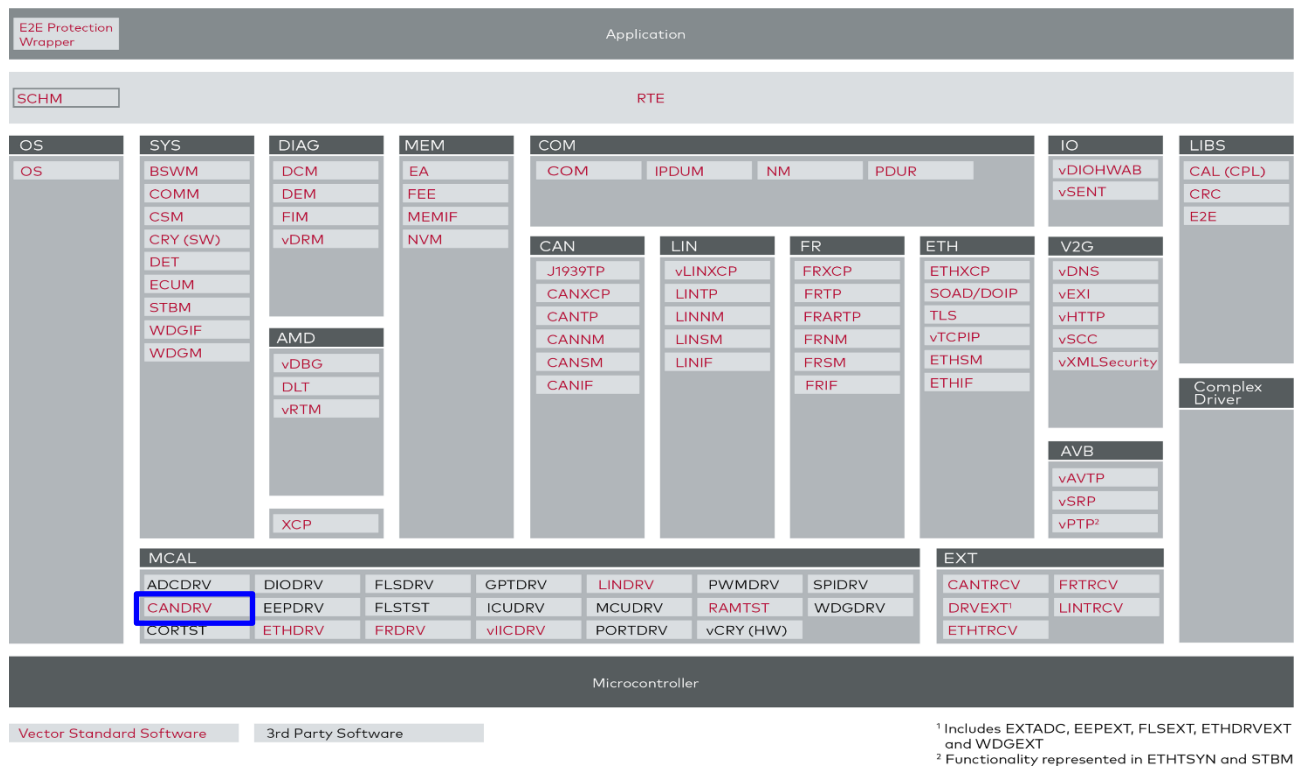


Figure 3-1 AUTOSAR architecture

The next figure shows the interfaces to adjacent modules of the CAN. These interfaces are described in chapter 6.

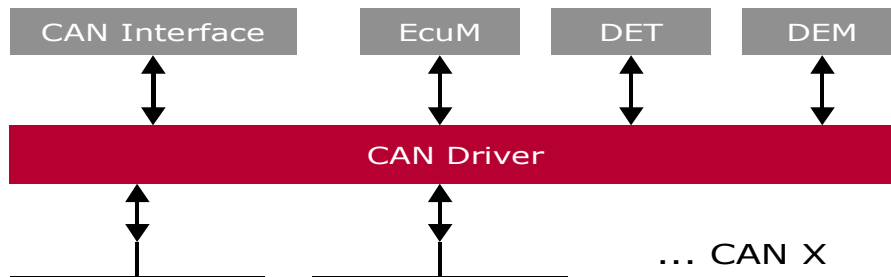


Figure 3-2 Interfaces to adjacent modules of the CAN

4 Functional Description

4.1 Features

The features listed in this chapter cover the complete functionality specified in [1].

The "supported" and "not supported" features are presented in the following table. For further information of not supported features also see chapter 8.

Feature Naming	Short Description	GENy	CFG5
Initialization			
Driver	General driver initialization function Can_Init()	■	■
Controller	Controller specific initialization function Can_InitController().	■	■
Communication			
Transmission	Transmitting CAN frames.	■	■
Transmit confirmation	Callback for successful Transmission.	■	■
Reception	Receiving CAN frames.	■	■
Receive indication	Callback for receiving Frame.	■	■
Controller Modes			
Sleep mode	Controller support sleep mode (power saving).	■	■
Wakeup over CAN	Controller support wakeup over CAN.	□	□
Stop mode	Controller support stop mode (passive to CAN bus).	■	■
Bus Off detection	Callback for Bus Off event.	■	■
Silent mode	Driver supports silent mode (listen only, passive to CAN bus).	□	■
Polling Modes			
Tx confirmation	Support polling mode for Transmit confirmation.	■	■
Reception	Support polling mode for Reception.	■	■
Wakeup	Support polling mode for Wakeup event.	□	□
Bus Off	Support polling mode for Bus Off event.	■	■
Mode	MICROSAR4x only: Support polling mode for mode transition.	■	■
Mailbox objects			
Tx BasicCAN	Standard mailbox to send CAN frames (Used by CAN Interface data queue).	■	■
Multiplexed Tx	Using 3 mailboxes for Tx BasicCAN mailbox (external priority inversion avoided).	■	■
Tx FullCAN	Separate mailbox for special Tx message used.	■	■

Maximum amount	Available amount of mailboxes per CAN cell.	63 - 255	63 - 255
Rx FullCAN	Separate mailbox for special Rx message used.	■	■
Maximum amount	Available amount of mailboxes per CAN cell.	63 - 255	63 - 255
Rx BasicCAN	Standard mailbox to receive CAN frame (depending on hardware, FIFO or shadow buffer supported).	■	■
Maximum amount	Available amount of BasicCAN objects.	63 - 255	63 - 255
Others			
DEM	Support Diagnostic Event Manager (error notification).	■	□***
DET	Support Development Error Detection (error notification).	■	■
Version API	API to read out component version.	■	■
Maximum supported controllers	Maximum amount of supported controllers (hardware channels) per CAN cell.	2 - 8	2 - 8
Cancellation of Tx objects	Support of Tx Cancellation (out of hardware). Avoid internal priority inversion.	□	□
Identical ID cancellation	Tx Cancellation also for identical IDs.	□	□
Standard ID types	Standard Identifier supported (Tx and Rx).	■	■
Extended ID types	Extended Identifier supported (Tx and Rx).	■	■
Mixed ID types	Standard and Extended Identifier supported (Tx and Rx).	■	■
CAN FD Mode1	FD frames with baudrate switch (BRS; TX and RX).	■	■
CAN FD Mode2	FD frames up to 64 data bytes (FULL support; TX and RX).	□	■****
Hardware Loop Check (Timeout monitoring)	To avoid possible endless loops (occur by hardware issue).	■	■
AutoSar extensions			
Individual Polling	Support individual polling mode (selectable for each mailbox separate).	■*	■*
Multiple Rx Basic CAN	Support Multiple Rx BasicCAN objects. This gives the possibility to use multiple Filters and optimize acceptance Filtering as well as avoid overrun.	■*	■*
Multiple Tx Basic CAN	Support Multiple Tx BasicCAN objects. Used to send different Tx groups over separate mailboxes with different buffering behavior (see CanInterface).	□	□
Rx Queue	Support Rx Queue. This give the possibility to buffer received data in interrupt context but handle it asynchronous in polling task.	■*	■*
Secure Rx Buffer used	Special hardware buffer used to temporary save received data.	■	■*****

Hardware Loop Check by Application	“Hardware Loop Check” can be defined to be done by application (special API available)	■	■
Configurable “Nested CAN Interrupts”	Nested CAN interrupts allowed, and can be also switched to none-nested. Nested means that higher prior interrupts may appear in the already active CAN interrupt context.	■	■
Report CAN_E_TIMEOUT DEM as DET	Report CAN_E_TIMEOUT (Hardware Loop Check / Timeout monitoring) to DET instead of DEM.	■	■
Support Mixed ID	Force CAN driver to handle Mixed ID (standard and extended ID) at pre-compile-time to expand the ID type later on.	■	■
Optimize for one controller	Activate this for 1 controller systems when you never will expand to multi-controller. So that the CAN driver works more efficient	■	■
Dynamic FullCAN Tx ID (***)	Always write FullCAN Tx ID within CanWrite() API function. Deactivate this to optimize code when you do not use FullCAN Tx objects dynamically.	■	□
Size of Hw HandleType	Support 8bit or 16bit Hardware Handles depend on hardware usage.	■	■
Generic PreCopy	Support a callback function for receiving any CAN message (following callbacks could be suppressed)	■	■
Generic Confirmation	Support a callback function for successful transmission of any CAN message (following callbacks could be suppressed)	■	■
Get Hardware Status	Support a API to get hardware status Information (see Can_GetStatus())	■	■
Interrupt Category selection	Support Category 1 or Category 2 Interrupt Service Routines for OS	■	■
Common CAN	Support merge of 2 controllers in hardware to get more Rx FullCAN objects	□	□
Overrun Notification	Support DET or Application notification cause by overrun (overwrite) of an Rx message (BasicCAN and FullCAN)	■	■
RAM check	Support CAN mailbox RAM check	■	■
Extended RAM Check	Support extended RAM check. Handling of individual deactivated mailboxes and controllers.	□	■
Multiple ECU configurations (***)	The feature Multiple ECU is usually used for nodes that exist more than once in a car. At power up the application decides which node should be realized.	■	□
Generic PreTransmit	Support a callback function with pointer to Data, right before this data will be written in Hardware mailbox buffer to send. (Use this to change data or cancel transmission)	■	■

Table 4-1 Supported features

- Feature is supported
- Feature is not supported
- * HighEnd Licence only
- ** Project specific (may not be available)
- *** Not supported or cannot be configured for MicroSar4
- **** Only available for MicroSar4
- ***** Not available if CAN FD Mode2 is used

4.2 Initialization

`Can_Init()` has to be called to initialize the CAN driver at power on and sets controller independent init values. This function has to be called before `Can_InitController()`.

MicroSar3 only: Use `Can_InitStruct()` to change the used baud rate and filter settings like given in the Initialization structure from the Tool. The used default set by `Can_InitMemory()` is the first structure. This API has to be called before `Can_InitController()` but after `Can_InitMemory()`.

MICROSAR401 only: baud rate settings given by `Can_InitController` parameter.

`Can_InitController()` initializes the controller, given as parameter, and can also be used to reinitialize. After this call the controller stays in stop-mode until the CAN Interface changes to start-mode.

`Can_InitMemory()` is an additional service function to reinitialize the memory to bring the driver back to a pre-power-on state (not initialized). Afterwards `Can_Init()` and `Can_InitController()` have to be called again. It is recommended to use this function before calling `Can_Init()` to secure that no startup-code specific pre-initialized variables affect the driver startup behavior.

4.3 Communication

`Can_Write()` is used to send a message over the mailbox object given as "Hth". The data, DLC and ID is copied into the hardware mailbox object and a send request is set. After sending the message the CAN Interface `CanIf_TxConfirmation()` function is called. Right before the data is copied in mailbox buffer the ID, DLC and data may be changed by `Appl_GenericPreTransmit()` callback.

When "Generic Confirmation" is activated the callback `Appl_GenericConfirmation()` will be called before `CanIf_TxConfirmation()` and the call to this can be suppressed by `Appl_GenericConfirmation()` return value.

For Tx messages the ID will be copied. (Exception: feature "Dynamic FullCAN Tx ID" is deactivated, then the FullCAN Tx messages will be only set while initialization)

If the mailbox is currently sending the status busy will be returned. Then the message may be queued in the CAN interface (if feature is active).

If cancellation in hardware is supported the lowest priority ID inside currently sending object is canceled, and therefore re-queued in the CAN Interface.

`Appl_GenericPreCopy()` (if activated) is called and depend on return value also `CanIf_RxIndication()` as a CAN Interface callback, is called when a message is received. The receive information like ID, DLC and data are given as parameter.

When Rx Queue is activated the received messages (polling or interrupt context) will be queued (same queue over all channels). The Rx Queue will be read by calling `Can_Mainfunction_Read()` and the Rx Indication (like `CanIf_RxIndication()`) will be called out of this context. Rx Queue is used for Interrupt systems to keep Interrupt latency time short.

4.3.1 Mailbox Layout

The generation tool supports a flexible allocation of message buffers. In the following tables the possible mailbox layout is shown (the range for each mailbox types depend on the used mailboxes).

Object number	Object type	No. of Objects	Comment
0 – n	Tx FullCAN	0-nmsg_left	These objects are used to send a certain message. The user must define statically (Generation Tool) which CAN messages are located in such Tx FullCAN objects. The Generation Tool distributes the messages to the FullCAN objects.
n+1	Tx BasicCAN	1 or 3 (depends on Multiplexed Tx feature)	These objects are used to send several messages. If the transmit message object is busy, the transmit request is stored in a CAN Interface queue (if activated)
n+2(5) – m (*)	Unused	0-nmsg_left	These objects are not used. It depends on the configuration of receive and transmit objects if unused objects are available.
m+1 – o	Rx FullCAN	0-nmsg_left	These objects are used to receive specific CAN messages. The user defines statically (Generation Tool) that a CAN message should be received in a FullCAN message object. The Generation Tool distributes the message to the FullCAN objects.
o+1 – o+2	Rx BasicCAN	0-nmsg_left	All other CAN messages are received via the BasicCAN message objects. Each BasicCAN message object consists of 2 message buffers (second used as Shadow buffer) and 1 Filter. If using "Multiple BasicCAN" feature the amount of used BasicCAN can be configured (also use this feature to select no BasicCAN), else there is always 1 or 2 BasicCAN objects.

Table 4-2 Hardware mailbox layout

`nmsg_left` : Amount of hardware mailboxes that are not allocated by other Object type. This is limited by derivative dependent maximum amount of mailboxes

*: for Cfg5/GeneratorMsr the Unused objects located behind Rx BasicCAN

The “CanObjectId” (ECUc parameter) numbering is done in following order: FullCAN Tx, BasicCAN Tx, Unused, FullCAN Rx, BasicCAN Rx (like shown above). “CanObjectId’s” for next controller begin at end of last controller. Gaps in “CanObjectId” for unused mailboxes may occur.



Expert Knowledge

For each CAN hardware object which is used with CAN FD mode, two additional hardware objects are configured by the generation tool. Take account to this if you configure your project.

If ‘Multiple CAN cells’ is used (physical channels from different cells are configured), two individual layouts are created, each reflecting the controllers that are mapped to the individual cell. The object numbers for the second cell start with 256.

4.3.2 Mailbox Processing Order

The hardware mailbox will be processed in following order:

Object Type	Order / priority to send or receive
Tx FullCAN	CAN ID Low to High
Tx BasicCAN	CAN ID Low to High
Rx FullCAN	Object ID Low to High
Rx BasicCAN	FIFO

In Case of Polling and Interrupt, FullCANs will be processed before BasicCANs.

The order between Rx and Tx mailboxes depends on the call order of the polling tasks or the interrupt context and cannot be guaranteed.

The Rx Queue will work like a FIFO filled with the above mentioned method.

4.3.3 Acceptance Filter for BasicCAN

The hardware acceptance filter of the BasicCAN object should be open only for BasicCAN messages. This reduces runtime because unused messages will not be handled. Each Basic CAN consist of one Filter (each FIFO element relates to that common filter). In case of mixed ID system two Basic CAN’s used one with standard-id and one with extended-id Filter.

If no message should be received, select the “Multiple Basic CAN” feature and set the amount to 0. Otherwise the filter should be set to “close”. Use feature “Rx BasicCAN Support” to deactivate unused code (for optimization).

4.3.4 Remote Frames

Remote Frames rejected in Hardware	Remote Frames rejected in Software
X	

Remote Frames will not have any influence on communication because they will not be received due to hardware filtering.

4.4 States / Modes

You can change the CAN cell mode via `Can_SetControllerMode()`. The last requested transition will be executed. The Upper layer has to take care about valid transitions.

Following modes changes are supported:

`CAN_T_START`

`CAN_T_STOP`

`CAN_T_SLEEP` (not supported – stop mode used instead)

`CAN_T_WAKEUP` (not supported – return to start mode)

MICROSAR4 only: Notification of mode change may occur asynchronous by notification `CanIf_ControllerModeIndication()`

4.4.1 Start Mode (Normal Running Mode)

This is the mode where communication is possible. This mode has to be set after Initialization because Controller is first in stop-mode.

4.4.2 Stop Mode

The Controller is set to “Init” mode so that no communication include Rx, TX and acknowledge will occur. The controller is disconnected from bus.

A hardware loop will be processed to re-initialize message buffers and reallocate it to controller lists see chapter “Hardware Loop”.

4.4.3 Sleep Mode

Sleep mode is only supported as emulated Sleep.

`EcuM_CheckWakeup()` is called in case a wakeup over CAN event occurred.

`Can_Cbk_CheckWakeup()` can be called to check wakeup state.

`CAN_T_WAKEUP` has to be set with `Can_SetControllerMode()` to leave this mode.

If Wakeup support is deactivated by configuration, the controller went into stop mode instead of sleep.

4.4.4 Bus Off

`CanIf_ControllerBusOff()` is called when the controller detects a Bus Off event. The mode is automatically changed to stop mode. The upper layers have to care about returning to normal running mode by calling start mode

4.4.5 Silent Mode

In Silent Mode (also called as Analyzer Mode in MultiCAN manual) the controller does not participate in the CAN communication. The transmission pin is connected to recessive level all the time. All messages including dominant bit, e.g. Ack message, error message, etc., will be suppressed. The controller could receive a message only when this message on bus is acknowledged by some other node on the same bus. A reception interrupt(or polling task) will come and serve only in this case.

4.5 Re-Initialization

A call to `Can_InitController()` cause a re-initialization of a dedicated CAN controller. Pending messages may be processed before the transition will be finished. A re-initialization is only possible out of Stop Mode and does not change to another Mode.

After re-initialization all CAN communication relevant registers are set to initial conditions.

4.6 CAN Interrupt Locking

`Can_DisableControllerInterrupts()` and `Can_EnableControllerInterrupts()` are used to disable and enable the controller specific Interrupt, Rx, Tx, Wakeup and Bus Off (/ Status) together. These functions can be called nested.

MICROSAR4 only: In case of “`CanUseOsInterruptControl == STD_ON`”, OS is called to handle interrupt locking, the CAN driver does not care for the Service Request Control registers (SRC register) at all.

4.7 Main Functions

`Can_MainFunction_Write()`, `Can_MainFunction_Read()`, `Can_MainFunction_BusOff()` and `Can_MainFunction_Wakeup()` called by upper layer to poll the events if the specific polling mode is activated. Otherwise these functions return without action and the events will be handled in interrupt context.

When individual polling is activated only mailboxes that are configured as to be polled will be polled in the main functions “`Can_MainFunction_Write()`” and “`Can_MainFunction_Read()`” all others handled in interrupt context.

When Rx Queue is activated the queue is filled in interrupt or polling context like configured. But The processing (indications) will be done in “`Can_MainFunction_Read()`” context.

MICROSAR4 only: `Can_MainFunction_Mode()` called by upper layer to poll asynchronous mode transition notifications.

4.8 Error Handling

4.8.1 Development Error Reporting

Development errors are reported to DET using the service `Det_ReportError()`, if the pre-compile parameter `CAN_DEV_ERROR_DETECT == STD_ON`.

The tables below, shows the API ID and Error ID given as parameter for calling the DET. Instance ID is always 0 because no multiple Instances are supported.

Errors reported to DET:	
Error ID	Short Description
CAN_E_PARAM_POINTER	API gets an illegal pointer as parameter.
CAN_E_PARAM_HANDLE	API get an illegal handle as parameter
CAN_E_PARAM_DLC	API get an illegal DLC as parameter
CAN_E_PARAM_CONTROLLER	API get an illegal controller as parameter
CAN_E_UNINIT	Driver API is used but not initialized
CAN_E_TRANSITION	Transition for mode change is illegal
CAN_E_DATA_LOST (value: 0x07, AutoSar extension)	Rx overrun (overwrite) detected
CAN_E_PARAM_BAUDRATE (value: 0x08, AutoSar extension)	Selected Baudrate is not valid
CAN_E_RX_QUEUE (value: 0x10, AutoSar extension)	Rx Queue overrun (last received message is lost, and will not be received → increase queue size)
CAN_E_TIMEOUT_DET (value: 0x11, AutoSar extension)	Same as CAN_E_TIMEOUT for DEM but this is notified to DET due to switch "CAN_DEV_TIMEOUT_DETECT" is set to STD_ON (see configuration options)
kErrorUnknownInterrupt (value: 0xa0, AutoSar extension)	CAN interrupt occur for unknown hardware mailbox.
kErrorUnknownISRPriority (value: 0xa1, AutoSar extension)	ISR priority has implausible value

Table 4-3 Errors reported to DET

API from which the errors reported to DET:	
API ID	Functions using that ID
CAN_VERSION_ID	Can_GetVersionInfo()
CAN_INIT_ID	Can_Init()
CAN_INITCTR_ID	Can_InitController()
CAN_SETCTR_ID	Can_SetControllerMode()
CAN_DIINT_ID	Can_DisableControllerInterrupts()
CAN_ENINT_ID	Can_EnableControllerInterrupts()
CAN_WRITE_ID	Can_Write(), Can_CancelTx()
CAN_TXCNF_ID	CanHL_TxConfirmation()
CAN_RXINDI_ID	CanBasicCanMsgReceived(), CanFullCanMsgReceived()
CAN_CTRBUSOFF_ID	CanHL_ErrorHandling()
CAN_CKWAKEUP_ID	CanHL_WakeUpHandling(), Can_Cbk_CheckWakeup()

CAN_MAINFCT_WRITE_ID	Can_MainFunction_Write()
CAN_MAINFCT_READ_ID	Can_MainFunction_Read()
CAN_MAINFCT_BO_ID	Can_MainFunction_BusOff()
CAN_MAINFCT_WU_ID	Can_MainFunction_Wakeup()
CAN_MAINFCT_MODE_ID	Can_MainFunction_Mode()
CAN_CHANGE_BR_ID	Can_ChangeBaudrate()
CAN_CHECK_BR_ID	Can_CheckBaudrate()
CAN_SET_BR_ID	Can_SetBaudrate()
CAN_HW_ACCESS_ID (value: 0x20, AUTOSAR extension)	Used when hardware is accessed (call context is unknown)

Table 4-4 API from which the Errors are reported

4.8.1.1 Parameter Checking

AUTOSAR requires that API functions check the validity of their parameters (Refer to [1]). These checks are for development error reporting and can be enabled and disabled separately. Refer to the configuration chapter where the enabling/disabling of the checks is described. Enabling/disabling of single checks is an addition to the AUTOSAR standard which requires enable/disable the complete parameter checking via the parameter `CAN_DEV_ERROR_DETECT`.

4.8.1.2 Overrun/Overwrite/FiFoFull Notification

The CAN driver serves two objects where Basic CAN messages may be received. The behavior of the Rx BasicCAN differs. Depending on if messages with a payload with more than 8 data bytes or less/equal 8 data bytes are configured to be received.

4.8.1.2.1 Rx message with more than 8 bytes payload

If more than the two available messages are received the CAN driver notifies an overrun.

Please note, if you get this overrun notification, the chronology of the received messages is not still valid.

4.8.1.2.2 Rx messages with less/equal 8 bytes payload

If the two available messages are received, the CAN driver notifies a FiFoFull.

Please note, if this notification is called, no message is lost yet. An overrun can't be detected in general.

As AUTOSAR extension the overrun detection may be activated by configuration tool. The notification can be configured to call an DET (MICROSAR4x) or Application call `Appl_CanOverrun()` or `Appl_CanFullCanOverrun()`.

4.8.2 Production Code Error Reporting

Production code related errors are reported to DEM using the service `Dem_ReportErrorStatus()`, if the pre-compile parameter `CAN_PROD_ERROR_DETECT == STD_ON`.

The table below shows the Event ID and Event Status given as parameter for calling the DEM. This callout may occur in the context of different API calls (see Chapter “Hardware Loop Check”).

Event ID	Event Status	Short Description
CAN_E_TIMEOUT	DEM_EVENT_STATUS_FAILED	Timeout in “Hardware Loop Check” occurred, hardware has to be checked or timeout is too short.

Table 4-5 Errors reported to DEM

4.8.2.1 Hardware Loop Check / Timeout Monitoring

The feature “Hardware Loop Check” is used to break endless loops caused by hardware issue. This feature is configurable see Chapter 7 and also Timeout Duration description.

The Hardware Loop Check will be handled by CAN driver internal except when setting “Hardware Loop Check by Application” is activated.

Loop Name / source	Short Description
kCanLoopListInit	- Waiting for hardware flag indication that the CAN message object list is correctly actualized (After change or reset). - Duration: Depend on CAN controller frequency and current activity of the CAN controller. (After re-init the ongoing CAN activities have to be finished before the list command could be ready). - One loop last about 20 cycles (depend on compiler optimization). - When the loop is left by break the loop (counter limit exceed). The Initialization has to be repeated to guarantee a correct behavior of CAN communication.
kCanLoopRx	- Loop while copy data from hardware object to temporary buffer to secure data consistency without locking receive mailboxes. The loop continue until received data is not overwritten. - Duration: when receiving function is delayed by higher task, there will be another loop because a message is received meanwhile overwriting the data in copy process. Due to the delay is shorter than on message length the loop should be executed only once. - When the loop is left by break the loop (counter limit exceed), the message will not be notified to upper layer.

Table 4-6 Hardware Loop Check

4.9 CAN RAM Check

The CAN driver supports a check of the CAN controller’s mailbox data registers (ID, DLC, data). The CAN controller RAM check is called internally every time a power on is executed within function `Can_InitController()`, or a Bus-Wakeup event happen. The CAN driver verifies that no used mailboxes are corrupt. A mailbox is considered corrupt if a predefined pattern is written to the appropriate mailbox registers and the read operation

does not return the expected pattern. If a corrupt mailbox is found the function `Appl_CanCorruptMailbox()` is called. This function tells the application which mailbox is corrupt.

After the check of all mailboxes the CAN driver calls the call back function `Appl_CanRamCheckFailed()` if at least one corrupt mailbox was found. The application must decide if the CAN driver disables communication or not by means of the call back function's return value. If the application has decided to disable the communication there is no possibility to enable the communication again until the next call to `Can_Init()` or a wakeup event occurs.

The CAN RAM check functionality itself can be activated via Generation Tool.

4.10 Extended RAM Check

The CAN driver supports a check for all accessible CAN controller's control registers and mailbox registers. The extended RAM check will be executed while power on initialization and by direct call. Mailboxes will be deactivated when pattern check or configured values are corrupt. The CAN controller will be deactivated when one mailbox is corrupt or the controller's registers failed the pattern check or configured values are corrupt. Mailbox and controller stay deactivated until they are explicit activated again.

API to execute extended RAM Check:

```
Can_RamCheckExecute()
```

Callouts to notify corrupt mailboxes or controllers:

```
CanIf_RamCheckCorruptController(), CanIf _RamCheckCorruptMailbox()
```

API to activate the mailbox or controller again:

```
Can_RamCheckEnableMailbox(), Can_RamCheckEnableController()
```

(See also API description)

The CAN RAM check functionality itself can be activated via Generation Tool.

4.11 Hardware Receive FIFO buffer

For Basic CAN message objects, with up to 8 databytes payload, it is possible to configure the depth of the hardware FIFO buffer.

For Basic CAN message objects with more than 8 databytes (CANFD), no FIFO is available. The Basic CAN message object is available as a shadow buffer with depth of 2 messages. If the first object received a message it will be locked until it will be preceded. If the first message object is locked, following Rx messages are received by the second message object. The second message object will be overwritten for each reception.

4.12 Hardware Specific

Currently no hardware specific issue is known. Please refer to errata sheets.

5 Integration

This chapter gives necessary information for the integration of the MICROSAR CAN into an application environment of an ECU.

5.1 Scope of Delivery

The delivery of the CAN contains the files, which are described in the chapter's 5.1.1 and 5.1.2:

Dependent on library or source code delivery the marked (+) files may not be delivered.

5.1.1 Static Files

File Name	Description
(+) Can_Local.h	This is an internal header file which should not be included outside this module
(+) Can.c	This is the source file of the CAN. It contains the implementation of CAN module functionality.
(+) Can.lib	This is the library build out of Can.c, Can.h and Can_Local.h
Can.h	This is the header file of the CAN module (include API declaration)
Can_Hooks.h	This is the header file to define the Hook-functions or macros. (this is a project specific file and may not exist)
Can_Irq.c	This is the interrupt declaration and callout file (supports interrupt configuration as link time settings)

Table 5-1 Static files

5.1.2 Dynamic Files

The dynamic files are generated by the configuration tool [GENy].

File Name	Description
Can_Cfg.h	Generated header file, contains some type, prototype and pre-compile settings
Can_Lcfg.c	Generated file contains link time settings.
Can_PBCfg.c	Generated file contains post build settings.
Can_DrvGeneralTypes.h	Generated file contains CAN driver part of Can_GeneralTypes.h (supported by Integrator)

Table 5-2 Generated files

5.2 Include Structure

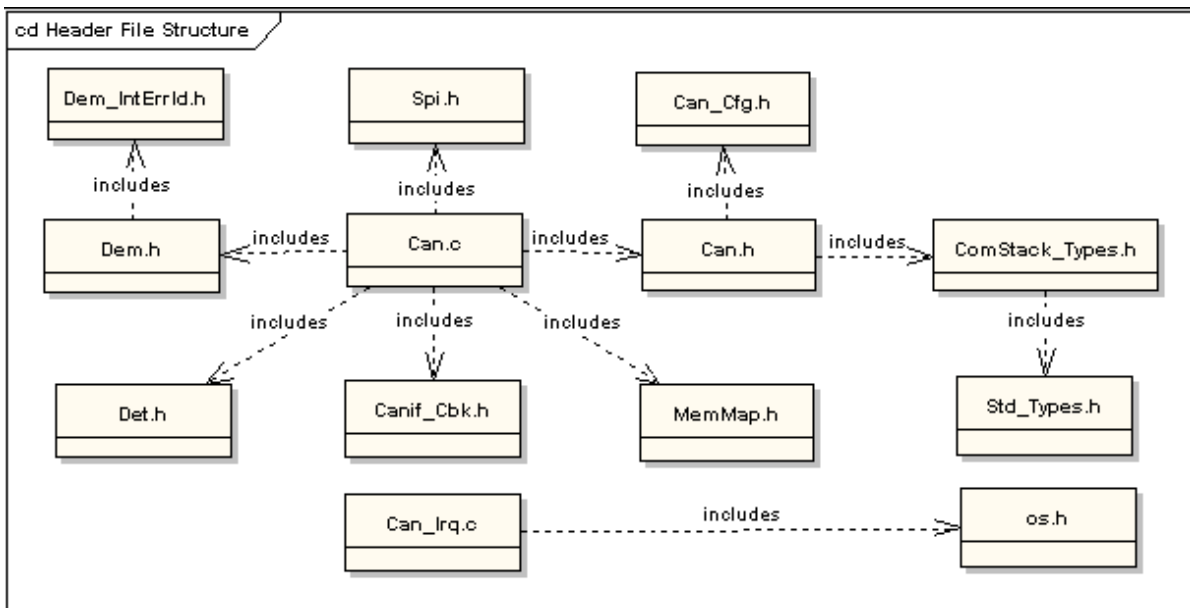


Figure 5-1 Include Structure (AUTOSAR)

Deviation from AUTOSAR specification:

- > Additionally the EcuM_Cbk.h is included by Can_Cfg.h (needed for wakeup notification API).
- > ComStack_Types.h included by Can_Cfg.h, because the specified types have to be known in generated data as well.
- > MICROSAR4x only: Os.h will be included by Can_Cfg.h because of used data-types
- > Spi.h is not yet used.
- > Additionally the file Can_Hooks.h may be included by Can.c.
- > MICROSAR403 only: Can_GeneralTypes.h will be included by Can_Cfg.h not by Can.h direct.

5.3 Critical Sections

The AUTOSAR standard provides with the BSW Scheduler a BSW module, which handles entering and leaving critical sections.

For more information about the BSW Scheduler please refer to [3]. When the BSW Scheduler is used the CAN Driver provides critical section codes that have to be mapped by the BSW Scheduler to following mechanism:

Critical Section Define	Description
CAN_EXCLUSIVE_AREA_0	Used inside ISR to ensure consistency of interrupt pending flags. > Duration is short (< 20 instructions) > No nesting of this AREA is allowed. Ensure that no other CAN ISR may occur nested.

CAN_EXCLUSIVE_AREA_1	Used inside Can_DisableControllerInterrupts() and Can_EnableControllerInterrupts() to avoid nested call of these APIs. > Duration is short (< 30 instructions) > No API call of other BSW inside. > Disable global interrupts – or – Empty in case Can_Disable/EnableControllerInterrupts() are called within context with lower or equal priority than CAN interrupt.
CAN_EXCLUSIVE_AREA_2	Used inside Can_Write() to secure software states of transmit objects. > Only when no Vector CAN Interface is used. > Duration is medium (< 100 instructions) > No API call of other BSW inside. > Disable global interrupts - or - Disable CAN interrupts and do not call function Can_Write() reentrant.
CAN_EXCLUSIVE_AREA_3	Used inside TX confirmation to secure state of transmit object in case of cancellation. (Only used when Vector Interface Version smaller 4.10 used) > Duration is medium (< 100 instructions) > Call to CanIf_CancelTxConfirmation() inside (no more calls in CanIf). > Disable global interrupts - or - Disable CAN interrupts and do not call function Can_Write() within.
CAN_EXCLUSIVE_AREA_4	Used inside received data handling (Rx Queue treatment) to secure Rx Queue counter and data. > Duration is short (< 30 instructions) > No API call of other BSW inside. > Disable Global Interrupts - or - Disable all CAN interrupts.
CAN_EXCLUSIVE_AREA_5	Used inside wakeup handling to secure state transition. (Only in wakeup polling mode) > Duration is short (< 20 instructions) > Call to DET inside. > Disable global interrupts (do not use CAN interrupt locks here)

CAN_EXCLUSIVE_AREA_6	Used inside Can_SetControllerMode() and BusOff to avoid nested state transition requests. Can_SetControllerMode() and BusOff-handling (polling or interrupt) must not be interrupted by other calls of Can_SetControllerMode(), BusOff-handling, Can_Init(), Can_InitController(), Can_ChangeBaudrate() or Can_SetBaudrate() > Duration is medium (< 100 instructions) > No API call of other BSW inside. > Use CAN interrupt locks here, in case the above mentioned APIs are only called within same tasklevel or CAN interrupt context. or Disable global interrupts
----------------------	---

Table 5-3 Critical Section Codes

5.4 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions defined for the CAN Interface and illustrates their assignment among each other.

Compiler Abstraction Definitions	CAN_CODE	CAN_STATIC_CODE	CAN_CONST	CAN_CONST_PBCFG	CAN_VAR_NOINIT	CAN_VAR_INIT	CAN_VAR_PBCFG	CAN_INT_CTRL	CAN_REG_CANCEL	CAN_RX_TX_DATA	CAN_APPL_CODE	CAN_APPL_CONST	CAN_APPL_VAR
CAN_START_SEC_CODE	■												
CAN_STOP_SEC_CODE													
CAN_START_SEC_STATIC_CODE		■											
CAN_STOP_SEC_STATIC_CODE													
CAN_START_SEC_CONST_8BIT			■										
CAN_STOP_SEC_CONST_8BIT													
CAN_START_SEC_CONST_16BIT			■										
CAN_STOP_SEC_CONST_16BIT													
CAN_START_SEC_CONST_32BIT			■										
CAN_STOP_SEC_CONST_32BIT													
CAN_START_SEC_CONST_UNSPECIFIED			■										
CAN_STOP_SEC_CONST_UNSPECIFIED													
CAN_START_SEC_PBCFG				■									
CAN_STOP_SEC_PBCFG													

CAN_START_SEC_PBCFG_ROOT				■									
CAN_STOP_SEC_PBCFG_ROOT													
CAN_START_SEC_VAR_NOINIT_UNSPECIFIED					■								
CAN_STOP_SEC_VAR_NOINIT_UNSPECIFIED													
CAN_START_SEC_VAR_INIT_UNSPECIFIED						■							
CAN_STOP_SEC_VAR_INIT_UNSPECIFIED													
CAN_START_SEC_VAR_PBCFG							■						
CAN_STOP_SEC_VAR_PBCFG													
CAN_START_SEC_CODE_APPL											■		
CAN_STOP_SEC_CODE_APPL													

Table 5-4 Compiler abstraction and memory mapping

The Compiler Abstraction Definitions CAN_ APPL_CODE, CAN_ APPL_VAR and CAN_ APPL_CONST are used to address code, variables and constants which are declared by other modules and used by the CAN driver.

These definitions are not mapped by the CAN driver but by the memory mapping realized in the CAN Interface or direct by application.

CAN_ CODE: used for CAN module code.

CAN_ STATIC_CODE: used for CAN module local code.

CAN_ CONST: used for CAN module constants.

CAN_ CONST_PBCFG: used for CAN module constants in Post-Build section.

CAN_ VAR_*: used for CAN module variables.

CAN_ INT_CTRL: is used to access the CAN interrupt controls.

CAN_ REG_CANCEL: is used to access the CAN cell itself.

CAN_ RX_TX_DATA: access to CAN Data buffers. (located in RAM area)

CAN_ APPL_*: access to higher layers.

5.5 Partial Network

It is not necessary to activate the partial network support via Pre-Configuration of the attribute: 'Can\CanGeneral\CanPnEnable'.

5.6 Hardware Specific Hints

The application is responsible for the initialization of the port pin control.

Initialization, which must be done before call of Can_Init():

- CAN port configuration (see also "IO Port" configuration):

Referring to the Infineon® Datasheet initialize the port registers to configure the needed IO-Port for CAN communication.

5.7 Multiple Can Driver

This is a special feature that makes it possible to support other CAN drivers working in parallel on the same MultiCAN cell. Enabling this feature leads to not initialized global CAN cell registers (like CLC, FDR). Also no List Register initialization will be accomplished by the CAN driver. This has to be done by the application.

Additional settings for the “Mailbox Start Index” and the “Used SRC” Registers are shown in the Generator Tool Channel View and have to be configured properly.

Do not activate this feature if there is no other CAN driver in parallel.

It is not supported to use hardware channels from different CAN cells (‘Multiple CAN cells’) in combination with the ‘Multiple Driver’ feature. Also the usage of the individual cells is not Post-BuildSelectable capable (meaning both cells are accessed in all variants, even if a variant does not use both cells).



Note

Sleep mode over CLC is no longer supported.

The CLC setting is expected to enable the Clock.

The FDR setting is expected to be performed according to the configuration



Caution

Ensure that “CAN_EXCLUSIVE_AREA_0” is not called nested.

When using this CAN driver on multiple cores then each core has to ensure that no CAN_EXCLUSIVE_AREA_0 from any other core will be interrupted.

Below you'll find an example code except (serving for a TC233 MultiCAN+) that illustrates the additional initialization effort which has to be accomplished by the application when using this feature:

Note: This example is serving for a TC233 MultiCAN+ and thus the address mapping has to be adapted to your derivative where required.

```
#define CANLL_CLC      *((volatile vuint32 *) 0xF0018000UL)
#define CANLL_FDR      *((volatile vuint32 *) 0xF001800CUL)
#define CANLL_PAN_CMD *((volatile vuint32 *) 0xF00181C4UL)
#define CANLL_NCR_BASE(ch) *((volatile vuint32 *) (0xF0018200UL + (ch*0x100UL)))
#define CANLL_MCR      *((volatile vuint32 *) 0xF00181C8UL)
void additionalStartupInitialization(void)
{ /* has to be called before calling Can_Init */

    volatile uint32 dummyRead;
    uint8 i;

    SCU_vResetENDINIT(0); /* unlock Safety ENDINIT protection */
    CANLL_CLC = 0x00000008UL; /* start CAN-clock, disable sleep */
    dummyRead = CANLL_CLC; /* check changes */
    CANLL_FDR = 0x00008300UL; /* set divider for baudrate-clock */
    SCU_vSetENDINIT(0); /* lock Safety ENDINIT protection */

    /* Reset CTRL and LIST fields of all message objects */
    while (CANLL_PAN_CMD & 0x00000100UL); /* wait while busy */
    CANLL_PAN_CMD = 0x00000001ul; /* initialize lists */
    while (CANLL_PAN_CMD & 0x00000100UL); /* wait while busy */

    /* Initialize all CAN nodes */
    for (i=0; i < CAN_MAX_HW_NODE_AMOUNT; i++)
    { /* De-activate all nodes, Re-activation takes place during Can_Init */
        CANLL_NCR_BASE(i) = 0x00000041UL; /* (CCE | INIT) */
    }

    /* Baudrate logic clock select */
    /* Note: with MultiCAN+ two steps are required:*/
    CANLL_MCR = 0; /* step 1: clear selection */
    ApplCanWaitSetMCR(); /* wait until cleared */
    CANLL_MCR = kCanMCRValue ); /* step 2: set new selection */
}
```

5.8 EndInit Protection

The CAN driver needs to access the EndInit protected registers CLC and FDR of the CAN-cell. Therefore the application must implement the Appl_UnlockEndinit() and Appl_LockEndinit() callback functions. Please refer to Infineons application notes and the Vector application note "AN-ISC-8-1172_TriCore_ENDINIT_protection_handling.pdf" how to implement this functions.

In certain cases the Vector AutoSar-Os provides the service to lock/unlock the Endinit protection. Please check this for the Vector OS documentation. If the API osClearENDINIT() and osSetENDINIT() is provided by the OS, include the header Os.h which sets the define:

```
#define osdEndInitAPISupported
```

Therewith the UnlockEndinit and LockEndinit is mapped to the OS.

5.9 Protected Register Access

If the protected register access is configured by the OS, you need to include the Os.h and configure the two areas:

CAN_PROTECTED_AREA_ENDINIT

CAN_PROTECTED_AREA_SRC (Aurix only)

The CAN driver will then not write directly to the SRC, CLC or FDR register, the OS carries this.

5.10 Nested CAN interrupts in conjunction with GNU compiler

With the option "Nested interrupts" it is possible to break the CAN interrupt service routine to serve higher prior interrupts.

For the Tricore, the global interrupts are disabled by hardware, if an interrupt service routine is entered. Disabled global interrupts are essential to secure the data consistency during the interrupt service routine for the CAN driver proceeds. If the GNU compiler is used, it is possible that the global interrupts are released while the interrupt service routine is entered.

For the reason of data consistency, it is essential to activate the option "Nested CAN interrupts" if your compiler releases the global interrupt lock on entering the interrupt service routine.



Caution

If this is not considered, it is possible that messages are not received/transmitted by random, respectively the communication stuck!

6 API Description

6.1 Interrupt Service Routines provided by CAN

Depend on the settings in Tools component Hw_TriCoreCpu, the interrupt routine is given by the driver or by Operating System. (Selection below, not MICROSAR403)

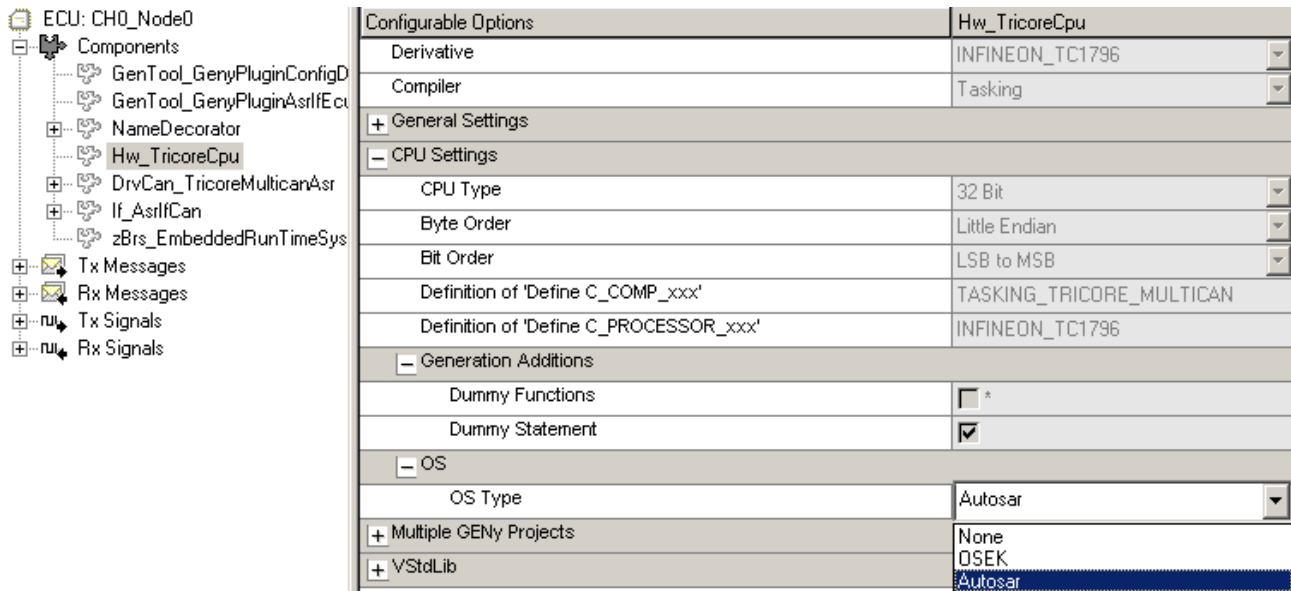


Figure 6-1 Select OS Type

There is the possibility to choose OS Type. Please select “None” for using no OS, “Autosar” for AUTOSAR OS or “OSEK” for OSEK OS systems.

6.1.1 OSEK (OS)

This means to include osek.h.

Switch: V_OSTYPE_OSEK

6.1.2 AutoSar (OS)

Os.h header file is used.

Switch: V_OSTYPE_AUTOSAR

6.1.3 None (OS)

Choose “None” for OS Type, to include no Os header files and have no category 2 interrupt.

Switch: V_OSTYPE_NONE

6.1.4 Type of Interrupt Function

See Chapter “Component Settings” for configuration aspects.

> Category 2 (only for OSEK OS or AUTOSAR OS):

A macro “ISR(CanIshr_x)” will be used to declare ISR function call. The name given as parameter for interrupt naming (x = Physical CAN Channel number). For macro definition see OS specification. The OS has full control of the ISR.

switch: C_ENABLE_OSEK_OS_INTCAT2



Caution

In case of Cat2 interrupts are used, the interrupt priority for each channel needs to be configured with the same priority level regarding OS and Can driver.

- > **Category 1:**
Using OS with category 1 interrupts need an Interface layer handling these interrupts in task context like defined in BSW00326 (AUTOSAR_SRS_General).
switch: C_DISABLE_OSEK_OS_INTCAT2
- > **Void-Void Interrupt Function:**
Like in Category 1 the Interrupt is not handled by OS and the ISR is declared as void ISR(void) and has to be called by interrupt controller in case of an CAN interrupt.
switch: C_ENABLE_ISRVOID

6.1.5 CanIsr_x

Prototype	
void CanIsr_x (void) → where x is the hardware controller index	
Parameter	
---	---
Return code	
---	---
Functional Description	
One Global Interrupt for each CAN channel is supported, handling Rx, TX and BusOff detection (only when not handled in polling)	
Particularities and Limitations	
> Not reentrant > If multiple CAN cells are incorporated in the used device, the hardware controller index for the first cell is in range 0..7 und 8..15 for the second cell.	

6.2 Services provided by CAN

The CAN API consists of services, which are realized by function calls.

6.2.1 Can_InitMemory

Prototype	
void Can_InitMemory (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Power-up memory initialization.	
Particularities and Limitations	
> Initializes component variables in *_INIT_* sections at power up. > Use this to re-run the system without performing a new start from power on. (E.g.: used to support an ongoing debug session without a complete re-initialization.)	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-1 Can_InitMemory

6.2.2 Can_Init

Prototype	
void Can_Init (Can_ConfigPtrType ConfigPtr)	
Parameter	
ConfigPtr [in]	Component configuration structure
Return code	
void	none
Functional Description	
Initializes component.	
Particularities and Limitations	
> Interrupts are disabled. > Module is uninitialized. > Can_InitMemory() has been called unless CAN Module is initialized by start up code. > Initializes all component variables and sets the component state to initialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-2 Can_Init

6.2.3 Can_InitController

Prototype	
void Can_InitController (uint8 Controller, Can_ControllerBaudrateConfigPtrType Config)	
Parameter	
Controller [in]	CAN controller
Config [in]	Pointer to baud rate configuration structure
Return code	
void	none
Functional Description	
Initialization of controller specific CAN hardware.	
Particularities and Limitations	
> Disabled Interrupts.Can_Init() has to be called. The CAN driver registers and variables are initialized. The CAN controller is fully initialized and left back within the state "STOP Mode", ready to change to "Running Mode". Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR40 Has to be called during the start up sequence before CAN communication takes place. Must not be called while in "SLEEP Mode". CREQ-969, CREQ-994	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-3 Can_InitController

6.2.4 Can_ChangeBaudrate

Prototype	
Std_ReturnType Can_ChangeBaudrate (uint8 Controller, const uint16 Baudrate)	
Parameter	
Controller [in]	CAN controller to be changed
Baudrate [in]	Baud rate to be set
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not set
Std_ReturnType	E_OK Baud rate is set

Functional Description
Change baud rate.
Particularities and Limitations
> The CAN controller must be in "STOP Mode". > This service shall change the baud rate and reinitialize the CAN controller. > Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && ((CAN_CHANGE_BAUDRATE_API == STD_ON) (CAN_SET_BAUDRATE_API == STD_OFF)) > Has to be called during the start up sequence before CAN communication takes place but after calling "Can_Init()". CREQ-995
Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 6-4 Can_ChangeBaudrate

6.2.5 Can_CheckBaudrate

Prototype	
Std_ReturnType Can_CheckBaudrate (uint8 Controller, const uint16 Baudrate)	
Parameter	
Controller [in]	CAN controller to be checked
Baudrate [in]	Baud rate to be checked
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not available
Std_ReturnType	E_OK Baud rate is available
Functional Description	
Checks baud rate.	
Particularities and Limitations	
> The CAN controller must be initialized. > This service shall check if the given baud rate is supported of the CAN controller. > Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && (CAN_CHANGE_BAUDRATE_API == STD_ON) CREQ-995	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-5 Can_CheckBaudrate

6.2.6 Can_SetBaudrate

Prototype	
Std_ReturnType Can_SetBaudrate (uint8 Controller, uint16 BaudRateConfigID)	
Parameter	
Controller [in]	CAN controller to be set
BaudRateConfigID [in]	Identity of the configured baud rate (available as Symbolic Name)
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not set
Std_ReturnType	E_OK Baud rate is set
Functional Description	
Set baud rate.	
Particularities and Limitations	
> The CAN controller must be in "STOP Mode". > This service shall change the baud rate and reinitialize the CAN controller. (Similar to Can_ChangeBaudrate() but used when identical baud rates are used for different CAN FD settings). > Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && (CAN_SET_BAUDRATE_API == STD_ON) CREQ-995	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-6 Can_SetBaudrate

6.2.7 Can_InitStruct

Prototype	
void Can_InitStruct (uint8 Controller, uint8 Index)	
Parameter	
Controller [in]	CAN controller to be changed
Index [in]	Index of the initialization structure to be used for baud rate and mask settings
Return code	
void	none
Functional Description	
Set active initialization structure.	
Particularities and Limitations	
> Can_Init() was called. > Set content of the initialization structure (before calling Can_InitController()).	

- > Service function to change the initialization structure set up left behind by the Generation Tool.
- > The structure contains information about baud rate and filter settings.
- > Subsequent Can_InitController() must be called to activate these settings.
- > Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR30 Call this function between calling "Can_Init()" and "Can_InitController()".
- > None AUTOSAR API CREQ-995

Call context

- > TASK
- > This function is Synchronous
- > This function is Non-Reentrant

Table 6-7 Can_InitStruct

6.2.8 Can_GetVersionInfo

Prototype

```
void Can_GetVersionInfo (Can_VersionInfoPtrType VersionInfo)
```

Parameter

VersionInfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
-------------------	--

Return code

void	none
------	------

Functional Description

Returns the version information.

Particularities and Limitations

- > Returns version information (MICROSAR3: BCD coded), vendor ID and AUTOSAR module ID (MICROSAR3: instance ID) of the component.
- > Configuration Variant(s): CAN_VERSION_INFO_API == STD_ON CREQ-991

Call context

- > ANY
- > This function is Synchronous
- > This function is Reentrant

Table 6-8 Can_GetVersionInfo

6.2.9 Can_GetStatus

Prototype

```
uint8 Can_GetStatus (uint8 Controller)
```

Parameter

Controller [in]	CAN controller requested for status information
-----------------	---

Return code	
uint8	CAN_STATUS_START START mode
	CAN_STATUS_STOP STOP mode
	CAN_STATUS_INIT Initialized controller
	CAN_STATUS_INCONSISTENT STOP or SLEEP are inconsistent over common CAN controllers
	CAN_DEACTIVATE_CONTROLLER RAM check failed CAN controller is deactivated
	CAN_STATUS_WARNING WARNING state
	CAN_STATUS_PASSIVE PASSIVE state
	CAN_STATUS_BUSOFF BUSOFF mode
	CAN_STATUS_SLEEP SLEEP mode
Functional Description	
Get status/mode of the given controller.	
Particularities and Limitations	
> Delivers the status of the hardware, as a bit coded value where multiple bits may be set. > Configuration Variant(s): CAN_GET_STATUS == STD_ON the return value can be analysed using the provided API macros: CAN_HW_IS_OK(), CAN_HW_IS_WARNING(), CAN_HW_IS_PASSIVE() CAN_HW_IS_BUSOFF(), CAN_HW_IS_WAKEUP(), CAN_HW_IS_SLEEP() CAN_HW_IS_STOP(), CAN_HW_IS_START(), CAN_HW_IS_INCONSISTENT() > None AUTOSAR API CREQ-978	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 6-9 Can_GetStatus

6.2.10 Can_SetControllerMode

Prototype	
Can_ReturnType Can_SetControllerMode (uint8 Controller, Can_StateTransitionType Transition)	
Parameter	
Controller [in]	CAN controller to be set
Transition [in]	Requested transition to destination mode CAN_T_START, CAN_T_STOP, CAN_T_SLEEP, CAN_T_WAKEUP
Return code	
Can_ReturnType	CAN_NOT_OK MICROSAR3: mode change unsuccessful - MICROSAR4: transition request rejected

Can_ReturnType	CAN_OK MICROSAR3: mode change successful - MICROSAR4: transition request accepted
Functional Description	
Change the controller mode.	
Particularities and Limitations	
> Interrupts locked > Request a mode transition witch processed synchronous for MICROSAR3 and return the success. > Or do it asynchronous for MICROSAR4 and call a notification when successful. BUSOFF, WAKEUP and RAM check will be handled as well.	
Call context	
> ANY > This function is Non-Reentrant	

Table 6-10 Can_SetControllerMode

6.2.11 Can_ResetBusOffStart

Prototype	
void Can_ResetBusOffStart (uint8 Controller)	
Parameter	
Controller [in]	CAN controller
Return code	
void	none
Functional Description	
Start BUSOFF handling.	
Particularities and Limitations	
> This is a compatibility function (for a CANbedded protocol stack) used during the start of the > BUSOFF handling to remove the BUSOFF state. > Configuration Variant(s): CAN_BUSOFF_SUPPORT_API == STD_ON Has to be called inside BUSOFF notification. > Empty function when not configured. > None AUTOSAR API CREQ-984	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-11 Can_ResetBusOffStart

6.2.12 Can_ResetBusOffEnd

Prototype	
void Can_ResetBusOffEnd (uint8 Controller)	
Parameter	
Controller [in]	CAN controller
Return code	
void	none
Functional Description	
Finish BUSOFF handling.	
Particularities and Limitations	
> Has to be called after Can_ResetBusOffStart(). > This is a compatibility function (for a CANbedded protocol stack) used during the end of the > BUSOFF handling to remove the BUSOFF state. > Configuration Variant(s): CAN_BUSOFF_SUPPORT_API == STD_ON The call should be done 128*11 recessive CAN bit times later because the call may wait this time to handle BUSOFF. > No nesting of Can_SetControllerMode() and Can_ResetBusOffEnd() is allowed. > Empty function when not configured. > None AUTOSAR API CREQ-984	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-12 Can_ResetBusOffEnd

6.2.13 Can_Write

Prototype	
Can_ReturnType Can_Write (Can_HwHandleType Hth, Can_PduInfoPtrType PduInfo)	
Parameter	
Hth [in]	Handle of the mailbox intended to send the message
PduInfo [in]	Information about the outgoing message (ID, dataLength, data)
Return code	
Can_ReturnType	CAN_NOT_OK transmit request rejected
	CAN_OK transmit request successful
	CAN_BUSY transmit request could not be accomplished due to the controller is busy.
Functional Description	
Send a CAN message.	

Particularities and Limitations
> disabled CAN interrupts / interrupts locked. > copy data, DLC and ID to the send mailbox and request a transmission.
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 6-13 Can_Write

6.2.14 Can_CancelTx

Prototype	
void Can_CancelTx (Can_HwHandleType Hth, PduIdType PduId)	
Parameter	
Hth [in]	Handle of the mailbox intended to be cancelled.
PduId [in]	PDU identifier
Return code	
void	none
Functional Description	
Cancel TX message.	
Particularities and Limitations	
> Cancel the TX message in the hardware buffer (if possible) or mark the message as not to be confirmed in case of the cancellation is unsuccessful.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-14 Can_CancelTx

6.2.15 Can_CheckWakeup

Prototype	
Std_ReturnType Can_CheckWakeup (uint8 Controller)	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
Std_ReturnType	E_OK the given controller caused a WAKEUP before.
Std_ReturnType	E_NOT_OK the given controller caused no WAKEUP before.

Functional Description
Check if WAKEUP occur.
Particularities and Limitations
> Check the occurrence of WAKEUP events for the given controller (used as WAKEUP callback for higher layers).
Call context
> ANY
> This function is Synchronous
> This function is Non-Reentrant

Table 6-15 Can_CheckWakeup

6.2.16 Can_DisableControllerInterrupts

Prototype	
void Can_DisableControllerInterrupts (uint8 Controller)	
Parameter	
Controller [in]	CAN controller to disable interrupts for.
Return code	
void	none
Functional Description	
Disable CAN interrupts.	
Particularities and Limitations	
> Must not be called while CAN controller is in SLEEP mode. > Disable the CAN interrupt for the given controller (e.g. due to data consistency reasons).	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-16 Can_DisableControllerInterrupts

6.2.17 Can_EnableControllerInterrupts

Prototype	
void Can_EnableControllerInterrupts (uint8 Controller)	
Parameter	
Controller [in]	CAN controller to enable interrupts for.
Return code	
void	none

Functional Description
Enable CAN interrupts.
Particularities and Limitations
> Must not be called while CAN controller is in SLEEP mode. > Re-enable the CAN interrupt for the given controller (e.g. due to data consistency reasons).
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 6-17 Can_EnableControllerInterrupts

6.2.18 Can_MainFunction_Write

Prototype	
void Can_MainFunction_Write (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
TX message observation.	
Particularities and Limitations	
> Must not interrupt the call of Can_Write(). > CAN driver has to be initialized. > Polling TX events (confirmation, cancellation) for all controllers and all TX mailboxes to accomplish the TX confirmation handling (like CanInterface notification).	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-18 Can_MainFunction_Write

6.2.19 Can_MainFunction_Read

Prototype	
void Can_MainFunction_Read (void)	
Parameter	
void	none

Return code	
void	none
Functional Description	
RX message observation.	
Particularities and Limitations	
> CAN driver has to be initialized. > Polling RX events for all controllers and all RX mailboxes to accomplish the RX indication handling (like CanInterface notification). > Also used for a delayed read (from task level) of the RX Queue messages which were queued from interrupt context.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-19 Can_MainFunction_Read

6.2.20 Can_MainFunction_BusOff

Prototype	
void Can_MainFunction_BusOff (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
BUSOFF observation.	
Particularities and Limitations	
> CAN driver has to be initialized. > Service function to poll BUSOFF events for all controllers to accomplish the BUSOFF handling (like calling of CanIf_ControllerBusOff() in case of BUSOFF occurrence).	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-20 Can_MainFunction_BusOff

6.2.21 Can_MainFunction_Wakeup

Prototype	
void Can_MainFunction_Wakeup (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
WAKEUP observation.	
Particularities and Limitations	
> CAN driver has to be initialized. > Polling WAKEUP events for all controllers to accomplish the WAKEUP handling (like calling of "CanIf_SetWakeupEvent()" in case of WAKEUP occurrence).	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-21 Can_MainFunction_Wakeup

6.2.22 Can_MainFunction_Mode

Prototype	
void Can_MainFunction_Mode (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Mode transition observation.	
Particularities and Limitations	
> Polling Mode changes over all controllers. (This is handled asynchronous if not accomplished in "Can_SetControllerMode()") > Configuration Variant(s): MICROSAR4x only CREQ-978	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-22 Can_MainFunction_Mode

6.2.23 Can_RamCheckExecute

Prototype	
<code>void Can_RamCheckExecute (uint8 Controller)</code>	
Parameter	
Controller [in]	CAN controller to be checked.
Return code	
void	none
Functional Description	
Start checking the CAN cells RAM.	
Particularities and Limitations	
> Has to be called within STOP mode. > Check all controller specific and mailbox specific registers by write patterns and read back. Issue notification will appear in this context. > Configuration Variant(s): CAN_RAM_CHECK == CAN_EXTENDED CREQ-106641	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-23 Can_RamCheckExecute

6.2.24 Can_RamCheckEnableMailbox

Prototype	
<code>void Can_RamCheckEnableMailbox (Can_HwHandleType htrh)</code>	
Parameter	
htrh [in]	CAN mailbox to be reactivated.
Return code	
void	none
Functional Description	
Reactivate a mailbox after RamCheck failed.	
Particularities and Limitations	
> Has to be called within STOP mode after RamCheck failed (controller is deactivated). > Mailbox will be reactivated by clear deactivation flag. > Configuration Variant(s): CAN_RAM_CHECK == CAN_EXTENDED CREQ-106641	
Call context	
> TASK > This function is Synchronous	

> This function is Non-Reentrant

Table 6-24 Can_RamCheckEnableMailbox

6.2.25 Can_RamCheckEnableController

Prototype	
void Can_RamCheckEnableController (uint8 Controller)	
Parameter	
Controller [in]	CAN controller to be reactivated.
Return code	
void	none
Functional Description	
Reactivate CAN cells after RamCheck failed.	
Particularities and Limitations	
> Has to be called within STOP mode after RamCheck failed (controller is deactivated). > CAN cell will be reactivated by execute reinitialization. > Configuration Variant(s): CAN_RAM_CHECK == CAN_EXTENDED CREQ-106641	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-25 Can_RamCheckEnableController

6.2.26 CanInterrupt

Prototype	
void CanInterrupt (CAN_HW_CHANNEL_CANTYPE_ONLY)	
Parameter	
-	
Return code	
void	-
Functional Description	
The interrupt service routine CanIsr_x handles all CAN Interrupts sequentially on CAN hardware channel x.	
Particularities and Limitations	
> Called by Interrupt Handler.	
Call context	
> CAN ISR routine for handling CAN Controller Interrupts. > This function is Synchronous	

- > This function is Non-Reentrant
- > Availability: For not pure polling configurations

Table 6-26 CanInterrupt

6.2.27 Appl_GenericPrecopy

Prototype	
Can_ReturnType Appl_GenericPrecopy (uint8 Controller, Can_IdType ID, uint8 DataLength, Can_DataPtrType DataPtr)	
Parameter	
Controller [in]	CAN controller which received the message.
ID [in]	ID of the received message. In case of extended or mixed ID systems the highest bit (bit 31) is set to mark an extended ID. FD-bit will not be set at all.
DataLength [in]	Data length of the received message.
pData [in]	Pointer to the data of the received message.
Return code	
Can_ReturnType	CAN_OK Higher layer indication will be called afterwards (CanIf_RxIndication()).
Can_ReturnType	CAN_NOT_OK Higher layer indication will not be called afterwards.
Functional Description	
Common RX indication callback that will be called before message specific callback will be called.	
Particularities and Limitations	
> Application callback function which informs about all incoming RX messages including the contained data. > It can be used to block notification to upper layer. E.g. to filter incoming messages or route it for special handling. > Configuration Variant(s): CAN_GENERIC_PRECOPY == STD_ON "pData" is read only and must not be accessed for further write operations. > The parameter DataLength refers to the received data length by the CAN controller hardware. > Note, that the CAN protocol allows the usage of data length values greater than eight (CAN-FD). > Depending on the implementation of this callback it may be necessary to consider this special case (e.g. if the data length is used as index value in a buffer write access). None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-27 Appl_GenericPrecopy

6.2.28 Appl_GenericConfirmation

Prototype	
Can_ReturnType Appl_GenericConfirmation (PduIdType PduId)	
Parameter	
PduId [in]	Handle of the PDU specifying the message.
Return code	
Can_ReturnType	CAN_OK Higher layer confirmation will be called afterwards (CanIf_TxConfirmation()).
Can_ReturnType	CAN_NOT_OK Higher layer confirmation will not be called afterwards.
Functional Description	
Common TX notification callback that will be called before message specific callback will be called.	
Particularities and Limitations	
> Application callback function which informs about TX messages being sent to the CAN bus. It can be used to block confirmation or route the information to other layer as well. > Configuration Variant(s): CAN_GENERIC_CONFIRMATION == STD_ON "PduId" is read only and must not be accessed for further write operations. > None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-28 Appl_GenericConfirmation

6.2.29 Appl_GenericPreTransmit

Prototype	
void Appl_GenericPreTransmit (uint8 Controller, Can_PduInfoPtrType_var DataPtr)	
Parameter	
Controller [in]	CAN controller on which the message will be send.
DataPtr [in]	Pointer to a Can_PduType structure including ID, DataLength, PDU and data pointer.
Return code	
void	none
Functional Description	
Common transmit callback.	
Particularities and Limitations	
> Application callback function allowing the modification of the data to be transmitted (e.g.: add CRC). > Configuration Variant(s): CAN_GENERIC_PRETRANSMIT == STD_ON None AUTOSAR API	
Call context	

- > ANY
- > This function is Synchronous
- > This function is Non-Reentrant

Table 6-29 Appl_GenericPreTransmit

6.2.30 ApplCanTimerStart

Prototype	
void ApplCanTimerStart (CanChannelHandle Controller, uint8 source)	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
void	none
Functional Description	
Start time out monitoring.	
Particularities and Limitations	
> Service function to start an observation timer. > Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && ((CAN_HARDWARE_CANCELLATION == STD_ON) MICROSAR4) Please refer to chapter "Hardware Loop Check". > None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-30 ApplCanTimerStart

6.2.31 ApplCanTimerLoop

Prototype	
Can_ReturnType ApplCanTimerLoop (CanChannelHandle Controller, uint8 source)	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
Can_ReturnType	> CAN_NOT_OK when loop shall be broken (observation stops)

	> CAN_NOT_OK should only be used in case of a time out occurs due to a hardware issue. > After this an appropriate error handling is needed (see chapter Hardware Loop Check / Time out Monitoring).
Can_ReturnType	> CAN_OK when loop shall be continued (observation continues)
Functional Description	
Time out monitoring.	
Particularities and Limitations	
> Service function to check (against generated maximum loop value) whether a hardware loop shall be continued or broken. > Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && ((CAN_HARDWARE_CANCELLATION == STD_ON) MICROSAR4) Please refer to chapter "Hardware Loop Check". > None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-31 ApplCanTimerLoop

6.2.32 ApplCanTimerEnd

Prototype	
void ApplCanTimerEnd (CanChannelHandle Controller, uint8 source)	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
void	none
Functional Description	
Stop time out monitoring.	
Particularities and Limitations	
> Service function to to end an observation timer. > Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && ((CAN_HARDWARE_CANCELLATION == STD_ON) MICROSAR4) Please refer to chapter "Hardware Loop Check". > None AUTOSAR API	
Call context	
> ANY > This function is Synchronous	

> This function is Non-Reentrant

Table 6-32 ApplCanTimerEnd

6.2.33 ApplCanInterruptDisable

Prototype	
void ApplCanInterruptDisable (uint8 Controller)	
Parameter	
Controller [in]	CAN controller for the CAN interrupt lock.
Return code	
void	none
Functional Description	
CAN interrupt disabling by application.	
Particularities and Limitations	
> Disabling of CAN Interrupts by the application. E.g.: the CAN driver itself should not access the common Interrupt Controller due to application specific restrictions (like security level). Or the application like to be informed because of an CAN interrupt lock. called by Can_DisableControllerInterrupts(). > Configuration Variant(s): CAN_INTLOCK == CAN_APPL CAN_INTLOCK == CAN_BOTH None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-33 ApplCanInterruptDisable

6.2.34 ApplCanInterruptRestore

Prototype	
void ApplCanInterruptRestore (uint8 Controller)	
Parameter	
Controller [in]	CAN controller for the CAN interrupt unlock.
Return code	
void	none
Functional Description	
CAN interrupt restore by application.	
Particularities and Limitations	
> Re-enabling of CAN Interrupts by the application.	

E.g.: the CAN driver itself should not access the common Interrupt Controller due to application specific restrictions (like security level). Or the application like to be informed because of an CAN interrupt lock. called by Can_EnableControllerInterrupts().

- > Configuration Variant(s): CAN_INTLOCK == CAN_APPL || CAN_INTLOCK == CAN_BOTH None AUTOSAR API

Call context

- > ANY
- > This function is Synchronous
- > This function is Non-Reentrant

Table 6-34 ApplCanInterruptRestore

6.2.35 Appl_CanOverrun

Prototype

```
void Appl_CanOverrun (uint8 Controller)
```

Parameter

Controller [in]	CAN controller for which the overrun was detected.
-----------------	--

Return code

void	none
------	------

Functional Description

Overrun detection.

Particularities and Limitations

- > Called when an overrun is detected for a BasicCAN mailbox.
- > Alternatively a DET call can be selected instead of ("CanOverrunNotification" is set to "DET").
- > Configuration Variant(s): CAN_OVERRUN_NOTIFICATION == CAN_APPL None AUTOSAR API

Call context

- > ANY
- > This function is Synchronous
- > This function is Non-Reentrant

Table 6-35 Appl_CanOverrun

6.2.36 Appl_CanFullCanOverrun

Prototype

```
void Appl_CanFullCanOverrun (uint8 Controller)
```

Parameter

Controller [in]	CAN controller for which the overrun was detected.
-----------------	--

Return code

void	none
------	------

Functional Description
Overrun detection.
Particularities and Limitations
> Called when an overrun is detected for a FullCAN mailbox. > Alternatively a DET call can be selected instead of ("CanOverrunNotification" is set to "DET"). > Configuration Variant(s): CAN_OVERRUN_NOTIFICATION == CAN_APPL None AUTOSAR API
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 6-36 Appl_CanFullCanOverrun

6.2.37 Appl_CanCorruptMailbox

Prototype	
void Appl_CanCorruptMailbox (uint8 Controller, Can_HwHandleType hwObjHandle)	
Parameter	
Controller [in]	CAN controller for which the check failed.
hwObjHandle [in]	Hardware handle of the defect mailbox.
Return code	
void	none
Functional Description	
Mailbox notification in case of RAM check failure.	
Particularities and Limitations	
> Will notify the application (during Can_InitController()) about a defect mailbox within the CAN cell. > Configuration Variant(s): CAN_RAM_CHECK == CAN_NOTIFY_MAILBOX None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-37 Appl_CanCorruptMailbox

6.2.38 Appl_CanRamCheckFailed

Prototype	
uint8 Appl_CanRamCheckFailed (uint8 Controller)	
Parameter	
Controller [in]	CAN controller for which the check failed.

Return code	
uint8	CAN_DEACTIVATE_CONTROLLER deactivate the controller
uint8	CAN_ACTIVATE_CONTROLLER activate the controller
Functional Description	
CAN controller notification in case of RAM check failure.	
Particularities and Limitations	
> Will notify the application (during Can_InitController()) about a defect CAN controller due to a previous failed mailbox check. > The return value decide how to proceed with the initialization. > Configuration Variant(s): CAN_RAM_CHECK == CAN_NOTIFY_MAILBOX / CAN_NOTIFY_ISSUE None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-38 Appl_CanRamCheckFailed

6.2.39 ApplCanCorruptRegisters

Prototype	
void ApplCanCorruptRegisters (CAN_CHANNEL_CANTYPE_ONLY)	
Parameter	
channel [in]	channel on which the CAN cell check failed
Return code	
void	none
Functional Description	
CAN RAM check failed.	
Particularities and Limitations	
> The CAN RAM check during CanInit() failed > Configuration Variant(s): C_ENABLE_CAN_RAM_CHECK & C_ENABLE_CAN_RAM_CHECK_EXTENDED	
Call context	
> CanInit() > This function is Synchronous > This function is Non-Reentrant	

Table 6-39 ApplCanCorruptRegisters

6.2.40 ApplCanRxFiFoFull

Prototype	
void ApplCanRxFiFoFull (uint8 hwchannel)	
Parameter	
-	
Return code	
void	none
Functional Description	
Receive hardware FIFO is full.	
Particularities and Limitations	
> This function must be implemented by application. The application gets a notification that the Rx HW FiFo is full. > Configuration Variant(s): - None AUTOSAR API	
Call context	
> Basic CAN receive context > This function is Synchronous > This function is Non-Reentrant	

Table 6-40 ApplCanRxFiFoFull

6.2.41 Appl_UnlockEndinit

Prototype	
void Appl_UnlockEndinit (void)	
Parameter	
-	
Return code	
void	none
Functional Description	
Set the ENDINIT Control Bit.	
Particularities and Limitations	
> This function must be implemented by application. The callback function needs to set the ENDINIT Control Bit. > Configuration Variant(s): - None AUTOSAR API	
Call context	
> Call within controller initialization while the registers CLC and FDR need to be set > This function is Synchronous > This function is Non-Reentrant	

Table 6-41 Appl_UnlockEndinit

6.2.42 Appl_LockEndinit

Prototype	
void Appl_LockEndinit (void)	
Parameter	
-	
Return code	
void	none
Functional Description	
Reset the ENDINIT Control Bit.	
Particularities and Limitations	
> This function must be implemented by application. The callback function needs to reset the ENDINIT Control Bit. > Configuration Variant(s): - None AUTOSAR API	
Call context	
> Call within controller initialization while the registers CLC and FDR need to be set > This function is Synchronous > This function is Non-Reentrant	

Table 6-42 Appl_LockEndinit

6.2.43 Appl_EnableInt

Prototype	
void Appl_EnableInt (void)	
Parameter	
-	
Return code	
void	none
Functional Description	
Enable Global Interrupts.	
Particularities and Limitations	
> This function must be implemented by application. Called to enable Global Interrupts to support nesting of Interrupts in CAN Interrupt context. > Configuration Variant(s): - None AUTOSAR API	
Call context	
> CAN interrupt > This function is Synchronous > This function is Non-Reentrant	

Table 6-43 Appl_EnableInt

6.2.44 ApplCanWaitSetMCR

Prototype	
void ApplCanWaitSetMCR (void)	
Parameter	
-	
Return code	
void	none
Functional Description	
Generate a delay.	
Particularities and Limitations	
> This callback function is called between the first (reset to zero) and the second (define the new clock source) write to the Module Control Register (MCR) to switch to a distinct clock source (MCR.CLKSEL). The function has to assure that the required minimum delay time is compliant according to the formula described below: Beneath the PLL different clock sources are possible to allow an independent clocking of the baud rate logic. This offers the possibility to change the frequency of the CAN cell without effecting the CAN communication. > The reconfiguration of the clock source has to be done by using two writes: first a write of zero to the CLKSEL bit-field, then a second write defining the new clock source. Between the first and the second write a delay of minimum $4 * (1/f_A) + 2 * (1/f_{CLC})$ must be inserted by software, where f_A is the frequency being switched off with the first write. Exception: in case that the f_{CLC} is selected as a baud rate and bit timing logic clock (MCR.CLKSEL = 1), no delay cycles between the writes are necessary. In both cases, simply using one write defining the new clock source is not allowed. > Configuration Variant(s): - None AUTOSAR API	
Call context	
> CAN controller initialization > This function is Synchronous > This function is Non-Reentrant	

Table 6-44 ApplCanWaitSetMCR

6.2.45 Can_SetSilentMode

Prototype	
Std_ReturnType Can_SetSilentMode (uint8 Controller, Can_SilentModeType silentMode)	
Parameter	
Controller [in]	CAN controller to be set
silentMode [in]	Activate or deactivate the silent mode with CAN_SILENT_ACTIVE, CAN_SILENT_INACTIVE (Enumeration)

Return code	
Std_ReturnType	E_OK mode change successful.
Std_ReturnType	E_NOT_OK mode change failed.
Functional Description	
Activate and deactivate the silent mode. Switch to silent mode, as a listen only mode without ACK, and deactivate this mode again for regular communication.	
Particularities and Limitations	
> The CAN controller must be in stop mode. > Configuration Variant(s): CAN_SILENT_MODE == STD_ON	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-45 Can_SetSilentMode

6.3 Services used by CAN

In the following table services provided by other components, which are used by the CAN are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError (see "Development Error Reporting")
DEM	Dem_ReportErrorStatus (see "Production Code Error Reporting")
EcuM	EcuM_CheckWakeup This function is called when Wakeup over CAN bus occur. EcuM_GeneratorCompatibilityError This function is called during the initialization, of the CAN Driver if the Generator Version Check or the CRC Check fails. (see [5])
Application (optional non AUTOSAR)	Appl_GenericPrecopy Appl_GenericConfirmation Appl_GenericPreTransmit ApplCanTimerStart/Loop/End

Component	API
	Appl_CanRamCheckFailed, Appl_CanCorruptMailbox ApplCanInterruptDisable/Restore Appl_CanOverrun, Appl_CanFullCanOverrun For detailed description see Chapter 6.2
CANIF	CanIf_CancelTxNotification (non AUTOSAR) A special Software cancellation callback only used within Vector CAN driver CAN Interface bundle. CanIf_TxConfirmation Notification for a successful transmission. (see [4]) CanIf_CancelTxConfirmation Notification for a successful Tx cancellation. (see [4]) CanIf_RxIndication Notification for a message reception. (see [4]) CanIf_ControllerBusOff Bus Off notification function. (see [4]) CanIf_ControllerModeIndication MICROSAR4x only: Notification for mode successfully changed.
Os (MICROSAR4x)	OS_TICKS2MS_<counterShortName>() Os macro to get timebased ticks from counter. GetElapsedValue Get elapsed tick count. GetCounterValue Get tick count start. osDisableInterruptSource Disable the CAN interrupt in the interrupt controller by OS osEnableInterruptSource Disable the CAN interrupt in the interrupt controller by OS osClearPendingInterrupt Clear pending CAN interrupt in the interrupt controller by OS osIsInterruptSourcePending Get the CAN interrupt pending state osIsInterruptSourceEnabled get the CAN interrupt enable state

Table 6-46 Services used by the CAN

7 Configuration

For CAN driver the attributes can be configured with configuration Tool “GENy”

The CAN driver supports pre-compile, link-time and post-build configuration.

For post-build loadable systems, re-flashing the generated data can change some configuration settings.

For post-build and link-time configurations pre-compile settings are configured at compile time and therefore unchangeable at link or post-build time.

The following parameters are set by GENy configuration (see Chapter “Configuration with GENy”).

7.1 Pre-Compile Parameters

Some settings have to be available before compilation:

- > Version API (Can_GetVersionInfo() activation)
`#define CAN_VERSION_INFO_API STD_ON/STD_OFF`
- > DET (development error detection)
`#define CAN_DEV_ERROR_DETECT STD_ON/STD_OFF`
- > Hardware Loop Check (timeout monitoring)
`#define CAN_HARDWARE_CANCELLATION STD_ON/STD_OFF`
- > Polling modes: Tx confirmation, Reception, Wakeup, BusOff
`#define CAN_TX_PROCESSING CAN_INTERRUPT/ CAN_POLLING`
`#define CAN_RX_PROCESSING CAN_INTERRUPT/ CAN_POLLING`
`#define CAN_BUSOFF_PROCESSING CAN_INTERRUPT/ CAN_POLLING`
`#define CAN_WAKEUP_PROCESSING CAN_INTERRUPT/ CAN_POLLING`
`#define CAN_INDIVIDUAL_PROCESSING STD_ON/STD_OFF`
- > Multiplexed Tx (external PIA – by use multiple Tx mailboxes)
`#define CAN_MULTIPLEXED_TRANSMISSION STD_ON/STD_OFF`
- > Configuration Variant (define the configuration type when using post-build variant)
`#define CAN_ENABLE_SELECTABLE_PB`
- > Use Generic Precopy Function (None AUTOSAR feature)
`#define CAN_GENERIC_PRECOPY STD_ON/STD_OFF`
- > Use Generic Confirmation Function (None AUTOSAR feature)
`#define CAN_GENERIC_CONFIRMATION STD_ON/STD_OFF`
- > Use Rx Queue Function (None AUTOSAR feature)
`#define CAN_RX_QUEUE STD_ON/STD_OFF`
- > Used ID type (standard/extended or mixed ID format)
`#define CAN_EXTENDED_ID STD_ON/STD_OFF`
`#define CAN_MIXED_ID STD_ON/STD_OFF`

- > Usage of Rx and Tx Full and BasicCAN objects (deactivate only when not using and to save ROM and runtime consumption)
`#define CAN_RX_FULLCAN_OBJECTS STD_ON/STD_OFF`
`#define CAN_TX_FULLCAN_OBJECTS STD_ON/STD_OFF`
`#define CAN_RX_BASICCAN_OBJECTS STD_ON/STD_OFF`
- > Use Multiple BasicCAN objects
`#define CAN_MULTIPLE_BASICCAN STD_ON/STD_OFF`
- > Use Multiple BasicCAN Tx objects
`#define CAN_MULTIPLE_BASICCAN_TX STD_ON/STD_OFF`
- > Optimizations
`#define CAN_ONE_CONTROLLER_OPTIMIZATION STD_ON/STD_OFF`
`#define CAN_DYNAMIC_FULLCAN_ID STD_ON/STD_OFF`
- > Usage of nested CAN interrupts
`#define CAN_NESTED_INTERRUPTS STD_ON/STD_OFF`
- > Use Multiple ECU configurations
`#define CAN_MULTI_ECU_CONFIG STD_ON/STD_OFF`
- > Use RAM Check (verify mailbox buffers)
`#define CAN_RAM_CHECK x`
- > Use Overrun detection
`#define CAN_OVERRUN_NOTIFICATION x`
- > Select MicroSar version
`#define CAN_MICROSAR_VERSION`
CAN_MSR403/CAN_MSR40/CAN_MSR30
- > Tx Cancellation of Identical IDs
`#define CAN_IDENTICAL_ID_CANCELLATION STD_ON/STD_OFF`
- > Module Start Address (only for post-build systems: The memory location for re-flashed data has to be defined)
- > Usage of Secure Temporary buffer

7.2 Link-Time Parameters

The library version of the CAN driver use following generated settings:

- > Maximum amount of used controllers and Tx mailboxes (has to be set for post-build variants at linktime)
- > Rx Queue size
- > Controller mapping (mapping of logical channel to hardware node).
- > Port selection Register (mapping of Rx IO ports)
- > SRC Register usage

7.3 Post-Build Parameters

Following settings are post-build data that can be changed for re-flashing:

- > Amount and usage of FullCAN Rx and Tx mailboxes
- > Used database (message information like ID, DLC)
- > Filters for BasicCAN Rx mailbox
- > Baud-rate settings
- > Module Start Address (only for post-build loadable systems: The memory location for re-flashed data has to be defined)
- > Configuration ID (only for post-build systems: This number is used to identify the post-build data)

7.4 Configuration with GENy

The CAN driver is configured with the help of the configuration tool GENy.

To generate communication specific settings, like used message objects e.g. FullCAN Rx mailboxes, baud rate settings and hardware specific settings for CAN communication, the GENy tool is used. This chapter explains the usage of this tool.

Please refer to the online help of the tool for detailed information about the general usage. And see below for special settings of the hardware specifics.

7.4.1 Platform Settings

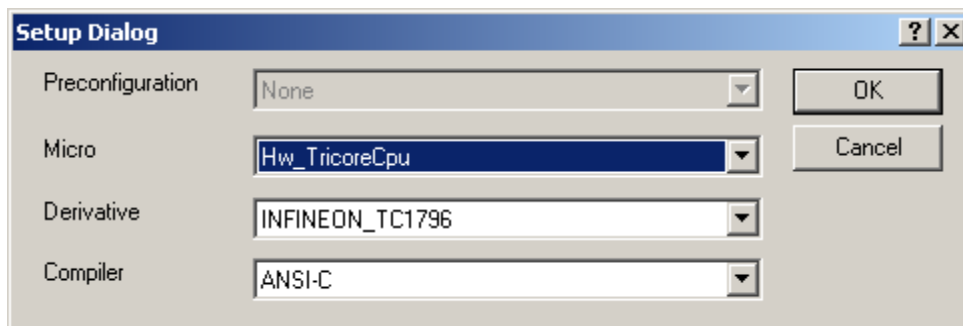


Figure 7-1 Platform settings

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
Micro	pre-compile	Enum	Hw_TricoreCpu	Select the hardware platform to use
Derivative	pre-compile	Enum	Name of the derivative	TC1XXX is a generic platform where the controller and maximum amount of mailboxes can be chosen.
Compiler	pre-compile	Enum	ANSI-C, Tasking,	Compiler selection

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
			GHS, DIABDATA	

Table 7-1 Platform Parameter description

7.4.2 Component Settings

ECU: CH1_Node0

- Components
 - Can_TricoreMultican
 - CanIf
 - GenTool_GenyAsrBase
 - GenTool_GenyPluginAsrIfEcu
 - GenTool_GenyPluginConfigD
 - Hw_TricoreCpu
 - NameDecorator
 - zBrs_EmbeddedRunTimeSyst
- Tx Messages
- Rx Messages
- Tx Signals
- Rx Signals

Configurable Options	Can_TricoreMultican
Multiple Can Driver	
Enable Multiple Can Driver	☐ *
Miscellaneous	
Hardware Loop Check	☒
Multiplexed Transmission	☒
Enable Wakeup Support	☐ *
Identical ID cancellation	☐ *
Divider Configuration	
CAN Clock Divider Mode	FRACTIONAL
CAN Clock Divider	768
Enable Multiple Divider	☐ *
Timeout	
Timeout Duration Factor	0x1388
Polling	
Tx Processing	Polling
Rx Processing	Polling
Busoff Processing	Interrupt
AUTOSAR extension	
Generic Precopy	☒
Generic Confirmation	☒
Rx FullCAN Support	☒ *
Tx FullCAN Support	☒ *
Rx BasicCAN Support	☒ *
Use Nested CAN Interrupts	☒ *
Secure RX Buffer Used	☒
Hardware Loop Check by Application	☐ *
Report CAN_E_TIMEOUT DEM as DET 0x07 notification	☐ *
Individual Processing	☒
Support Mixed ID	☐ *
Type of Interrupt Function	Category 1
Get Status API	☒
CAN Interrupt lock	Both
Multiple BasicCAN Objects	☒
Optimize for one controller	☐ *
Dynamic FullCAN Tx ID	☒ *
Size of Hw HandleType	8bit
Partial Networking	☐ *
RAM check	None
Overrun Notification	None
Generic PreTransmit	☒
Rx Queue	
Rx Queue	☐ *
Size	3*
Common	
Postbuild Configuration	

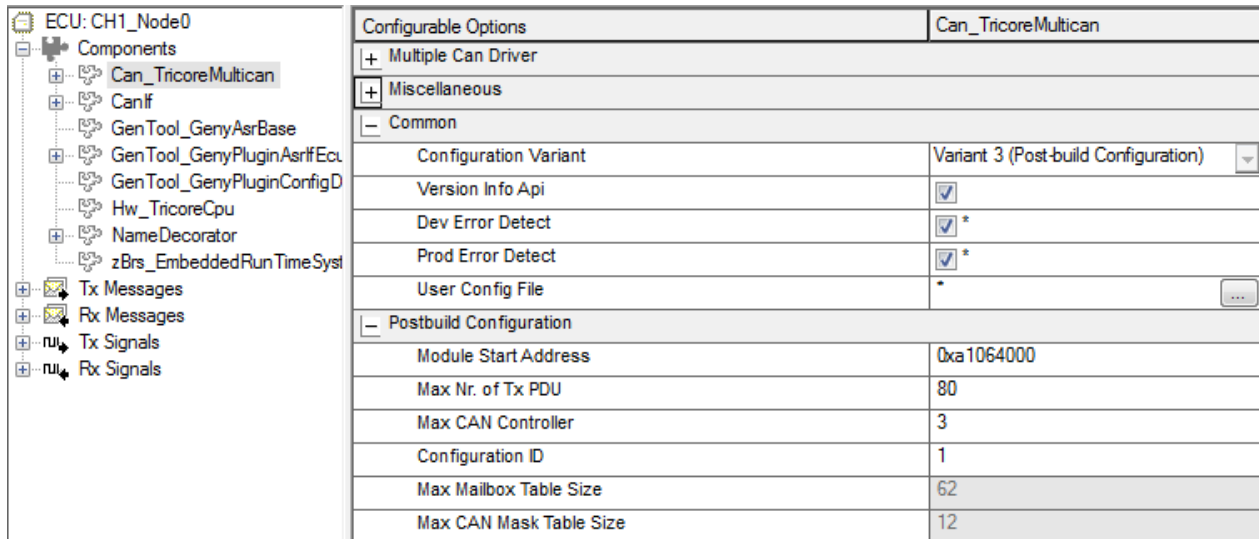


Figure 7-2 Component Settings

Attribute Name	Configuration Variant	Value Type	Values	Description
Multiple Can Driver				
Enable Multiple Can Driver	pre-compile	Bool	On / Off	This is a special feature that make it possible to support other CAN driver working in parallel on the MultiCAN cell. (Do not activate it if there is no other CAN driver in parallel). Enabling this feature leads to none initialized Global CAN cell registers like CLC, FDR and no List Registers reset will be processed. It also activates settings for mailbox start and used SRC registers on channel view. Sleep mode over CLC is no more supported. CLC is expected to enable the Clock. FDR is expected to be set according to configuration.
Divider Configuration				
CAN Clock Divider Mode	pre-compile	Enum	NORMAL / FRACTIONAL	Choose divider mode to calculate CAN clock (out of Sys clock). "Normal Divider Mode": $f_{CAN} = f_{SYS} \times 1/n$ "Fractional Divider Mode": $f_{CAN} = f_{SYS} \times n/1024$ with n = value of 'CAN Clock Divider' see below.
CAN Clock Divider	pre-compile	Integer	1-1023	Clock pre-divider for CAN cell to reduce fsys(refer to Datasheet)

Attribute Name	Configuration Variant	Value Type	Values	Description
Enable Multiple Divider	pre-compile	Bool	On / Off	Allow to configure multiple Divider settings (each set is assigned to the "Init Structure" with the same index and can be changed by switch "Init Structure")
Miscellaneous				
Hardware Loop Check	pre-compile	Bool	On / Off	Timeout monitoring to check for hardware problems inside a loop (possible endless loops because of hardware issues will be notified over DEM)
Timeout Duration Factor (MICROSAR3 only)	pre-compile	Integer	Loop count	Maximum loop count for "Hardware Loop Check". Refer to loop descriptions set this value for your needs.
Timeout Duration (MICROSAR4x only)	pre-compile	Float	Time	Specifies the time (in seconds) for the "Hardware Loop Check".
Timeout Counter ID (MICROSAR401 only)	pre-compile	Float	Time	Specifies the counter ID used in OS for 'Timeout Duration'
Multiplexed Transmission	pre-compile	Bool	On / Off	Activate to get multiplexed transmit objects for Tx BasicCAN object (3 send objects instead of 1) avoids external Id priority inversion
Wakeup Support	pre-compile	Bool	On / Off	Activate to Wake up over CAN bus by CAN driver
Hardware (Transmit) Cancellation	pre-compile	Bool	On / Off	Enable/disable the usage of 'transmit cancellation' out of hardware mailboxes, so that the canceled message won't be send. (avoid internal priority inversion)
Identical ID cancellation	pre-compile	Bool	On / Off	'Hardware Transmit Cancellation' also cancel messages with same identifier (newer data will be send) out of hardware mailboxes. otherwise only lower prior identifiers will be cancelled. (older data will be send)
Polling				
Tx Processing	pre-compile	Enum	Interrupt Polling /	Activate to handle transmit messages over polling
Rx Processing	pre-compile	Enum	Interrupt Polling /	Activate to handle receive messages over polling
Busoff Processing	pre-compile	Enum	Interrupt Polling /	Activate to handle Bus Off event over polling
Wakeup Processing	pre-compile	Enum	Interrupt Polling /	Activate to handle Wakeup event over polling

Attribute Name	Configuration Variant	Value Type	Values	Description
Mainfunction mode period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_Mode() in seconds
Mainfunction bus off period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_BusOff() in seconds
Mainfunction wakeup period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_Wakeup() in seconds
Mainfunction read period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_Read() in seconds
Mainfunction write period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_Write() in seconds
AUTOSAR extension				
Generic PreCopy	pre-compile	Bool	On / Off	Generic Rx notification for all messages
Generic Confirmation	pre-compile	Bool	On / Off	Generic Tx confirmation for all messages
Generic PreTransmit	pre-compile	Bool	On / Off	A generic PreTransmit, common for all Tx messages will be called when this checkbox is enabled. Use this to change data or abort transmission right before the message will be send.
Rx FullCAN Support	pre-compile	Bool	On / Off	Use Rx FullCAN message objects (deactivate to reduce ROM and runtime consumption)
Tx FullCAN Support	pre-compile	Bool	On / Off	Use Tx FullCAN message objects (deactivate to reduce ROM and runtime consumption)
Rx BasicCAN Support	pre-compile	Bool	On / Off	Use Rx BasicCAN message objects (deactivate to reduce ROM and runtime consumption)
Use Nested CAN Interrupts	pre-compile	Bool	On / Off	CAN ISRs from different controllers can interrupt each other (only with higher priority). deactivate to reduce runtime consumption.
Secure Rx Buffer used	pre-compile	Bool	On / Off	Temporary buffer for received data will be located in a unused mailbox object. This is to secure receive data by not locate it to common RAM.

Attribute Name	Configuration Variant	Value Type	Values	Description
Hardware Loop Check by Application	pre-compile	Bool	On / Off	Enable this when you like to handle "Hardware Loop Check" by your own. Using API functions "ApplCanTimerStart()", „ApplCanTimerEnd()" and „ApplCanTimerLoop()"
Report CAN_E_TIMEOUT DEM as DET 0x07 notification	pre-compile	Bool	On / Off	Enable this when you like to report CAN_E_TIMEOUT ("Hardware Loop Check") to DET instead of DEM (does only work when DET is activated)
Individual Processing	pre-compile	Bool	On / Off	Enable this when you like to configure mailbox specific polling/interrupt processing. This feature is only available with "High End" license.
Support Mixed ID	pre-compile	Bool	On / Off	Force CAN driver to handle Mixed ID (standard and extended ID) at pre-compile-time to expand the ID type later on.
Multiple BasicCAN Objects	pre-compile	Bool	On / Off	Support Multiple BasicCAN objects. This gives the possibility to use multiple Filters and optimize acceptance Filtering as well as avoid overrun. This feature is only available with "High End" license.
Optimize for one controller	pre-compile	Bool	On / Off	Activate this for 1 controller systems when you never will expand to multi-controller. So that the CAN driver works more efficient
Dynamic FullCAN Tx ID	pre-compile	Bool	On / Off	Always write FullCAN Tx ID within Can_Write() API function. Deactivate this to optimize code when you do not use FullCAN Tx objects dynamically.
Size of Hw HandleType	pre-compile	Enum	8bit 16bit	Support 8bit Hardware Handles as define in AutoSar 3.01 specified. Or change to 16bit like specified in AutoSar 4.0.
Partial Networking	pre-compile	Bool	On / Off	Activate this when you like to support a partial network that will be woken up by special NM message. This feature have to be enabled if one of the controllers will wake up by partial network transceiver.

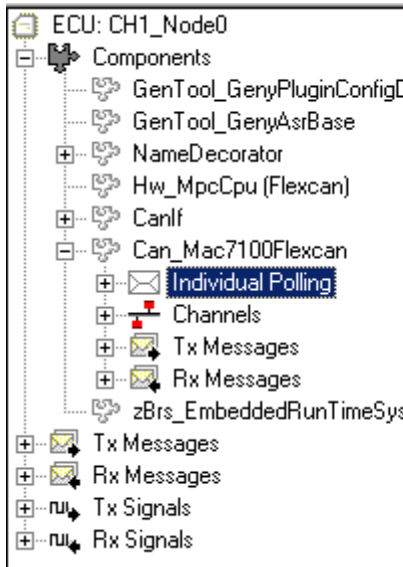
Attribute Name	Configuration Variant	Value Type	Values	Description
Get Status API	pre-compile	Bool	On / Off	Support Can_GetStatus() API that return the hardware states: CAN_STATUS_STOP, CAN_STATUS_INIT, CAN_STATUS_INCONSISTENT, CAN_STATUS_WARNING, CAN_STATUS_PASSIVE, CAN_STATUS_BUSOFF, CAN_STATUS_SLEEP
Type of Interrupt Function	Link-time	Enum	Category 1 Category 2 VoidFuncVoid	Category 1: Interrupt Function has to be added to the interrupt vector table unless this is not already done automatically by a compiler pragma. Category 2: Interrupt Function is defined with ISR() define. VoidFuncVoid: Interrupt Function is declared as void ISR(void) and has to be called in case of a CAN Interrupt.
CAN Interrupt lock	pre-compile	Enum	CANDriver Application Both	Disable and Restore Can Interrupt by CAN Driver or Application or CAN Driver and Application. Reason maybe, that the CAN driver should not access the Interrupt controller for CAN interrupts or there are other restrictions (special security level) or Application has to care about additional Interrupts like Wakeup.
Overrun Notification	pre-compile	Enum	None DET Application	Overrun detection activation and selection of the Notification function. None: No Overrun detection. DET: will be called for Overrun. Application: will be called.
RAM check	pre-compile	Enum	None Active Mailbox Notification	RAM check of mailbox buffers None: No RAM check Active: Active RAM check (global notification) Mailbox Notification: Active RAM check (global + mailbox notification)
Rx Queue				
Rx Queue	pre-compile	Bool	On / Off	Enable this when you like to handle Rx messages in polling context. (Shorten interrupt latency time). This feature is only available with "High End" license.

Attribute Name	Configuration Variant	Value Type	Values	Description
Size	pre-compile	Bool	On / Off	Size of FIFO in which the Message information is buffered.
TriCore MultiCAN (CPU)				
Number of CAN Objects (only "generic derivative")	pre-compile	Integer	Amount	Amount of CAN channels available for this derivative. Get value from your hardware manual. RESTRICTIONS: This value is only used for Generic Derivative
Maximum number of hardware CAN channels (only "generic derivative")	pre-compile	Integer	Amount	Amount of CAN mailbox objects available for this derivative. (over all channels) Get value from your hardware manual. RESTRICTIONS: This value is only used for Generic Derivative
Clock Source Selection (only "MultiCAN+ can cell")	pre-compile	Enum	On / Off	Beneath the PLL different clock sources are possible to allow an independent clocking of the baud rate logic. This offers the possibility to change the frequency of the CAN cell without effecting the CAN communication. The following clock sources are enabled today for the baudrate logic: - not used: disable the clock of the baudrate logic (with this reset setting no communication is possible) - f CLC: PLL - f Osc: Direct oscillator drive - ERAY PLL: can be used for FlexRay devices - hard wired "0": disable the clock of the baudrate logic
CAN Cell Base Address (only "generic derivative")	pre-compile	Integer	Address	CAN cell base address for MultiCAN cell please refer to derivative specific hardware manual.
CAN SRC Address (only "MultiCAN+ can cell")	pre-compile	Integer	Address	Address of first SRC register for CAN cell purpose. Get value from derivative specific hardware manual.
Common				
Configuration Variant	pre-compile	Enum	Pre-Compile, Link-Time, Post build	Select your project type
Version Info API	pre-compile	Bool	On / Off	Activate function to get version information

Attribute Name	Configuration Variant	Value Type	Values	Description
Dev Error Detection	pre-compile	Bool	On / Off	Activate to get information about possible conflicts (refer to DET module description)
Prod Error Detection	pre-compile	Bool	On / Off	Activate to get information about possible conflicts in a running system (refer to DEM module description)
User Config File	pre-compile	String	File name and path	This file will be included in the Can_cfg.h for special settings
Post build loadable configuration				
Module Start Address (Post-build only)	post-build	Integer	Address	Address to Flash the data
Max Nr. Of TX Objects (Post-build only)	Link-time	Integer	Amount	Amount of maximum used transmit objects over all controllers (used for RAM variables). One object for BasicCAN TX and one for each FullCAN TX. Add all objects over all used Channels for this value.
Max CAN Controller (Post-build only)	Link-time	Integer	Amount	Amount of maximum used controllers (used for RAM variables)
Configuration ID (Post-build only)	post-build	Integer	Identity	A special value to identify this configuration in the Flash.

Table 7-2 Component Parameter description

7.4.3 Individual Polling Settings



	Object Type	Channel	Polling
RxMSG00000435_1	FullCAN Rx	DUT_CH1	☐ *
RxMSG00000434_1	FullCAN Rx	DUT_CH1	☐ *
RxMSG00000135_1	FullCAN Rx	DUT_CH1	☒
RxMSG00000134_1	FullCAN Rx	DUT_CH1	☒
RxMSG00000133_1	FullCAN Rx	DUT_CH1	☐ *
BasicCAN0 Rx Polling	BasicCAN Rx	DUT_CH1	☐ *
BasicCAN1 Rx Polling	BasicCAN Rx	DUT_CH1	☒
Normal Tx Polling	Normal Tx	DUT_CH1	☐ *
TxMSG00000141_1	FullCAN Tx	DUT_CH1	☒
TxMSG00000142_1	FullCAN Tx	DUT_CH1	☒
TxMSG00000243_1	FullCAN Tx	DUT_CH1	☐ *
TxMSG00000444_1	FullCAN Tx	DUT_CH1	☐ *
TxMSG00000445_1	FullCAN Tx	DUT_CH1	☐ *

Figure 7-2 Individual polling

When “Individual Polling” feature in 7.4.2 is selected you can select for each mailbox Rx/Tx event to be polled or handled by interrupt. In case of FullCAN mailbox the name of

the message is shown. In case of BasicCAN mailbox “BasicCAN Rx (x)” or “Normal Tx” has to be selected to handle all messages related to this mailbox in polling mode.

“Normal Tx” is the collection of all none FullCAN Tx messages.

“BasicCAN Rx (X)” is the collection of all none FullCAN Rx messages pass the BasicCAN Filter “X”.

Figure 7-3 Common CAN channel setting

7.4.4 Controller (Channel) Settings

Figure 7-4 Controller Settings

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
Miscellaneous				
Physical Node	pre-compile	Enum	0-3	Check which node (0-3) has to be used for this CAN controller
IO Port	link-time	Enum	000 – 111	Initialization value of the Receive Input Selection for the selected Node in the register NPCR.RXSEL (CAN Node Port Control Register in binary format).
Interrupt Priority	link-time	Integer	0-255	Interrupt priority for the selected node
Multiple Can Driver				
SRC	link-time	Integer	0-15	Enabling “Enable Multiple Can Driver” activates this setting to select the used

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
				SRC register for this Node
Channel CanObject StartIndex	pre-compile	Integer	0-128	Enabling "Enable Multiple Can Driver" activates this setting to select the used hardware objects for this Node. Use Mailboxes begin from this start index up to the amount described in "Number of BasicCAN Objects"
AUTOSAR extension				
Common CAN	Link-time	Bool	On / Off	Activate this when you like to connect two physical CAN channels to one logical CAN Controller. This allows you to use more FullCAN for this Controller. Only the Rx FullCAN can be configured on second physical channel. RESTRICTIONS: 'CanCommonCAN' is a project specific feature and may is not available or supported for every platform.
Number of BasicCAN Objects	Link-time	Integer	Amount	Amount of used BasicCAN Objects. Increase this to optimize BasicCAN filtering. (need Multiple BasicCAN feature)
TriCore MultiCAN (CPU)				
Number of CAN Objects	post-build	Integer	0-128	Maximum number of hardware objects, which can be used by this controller. (depend on derivative, up to 128 objects available for all controllers)

Table 7-2 Controller Parameter description

7.4.5 Init Structure Settings

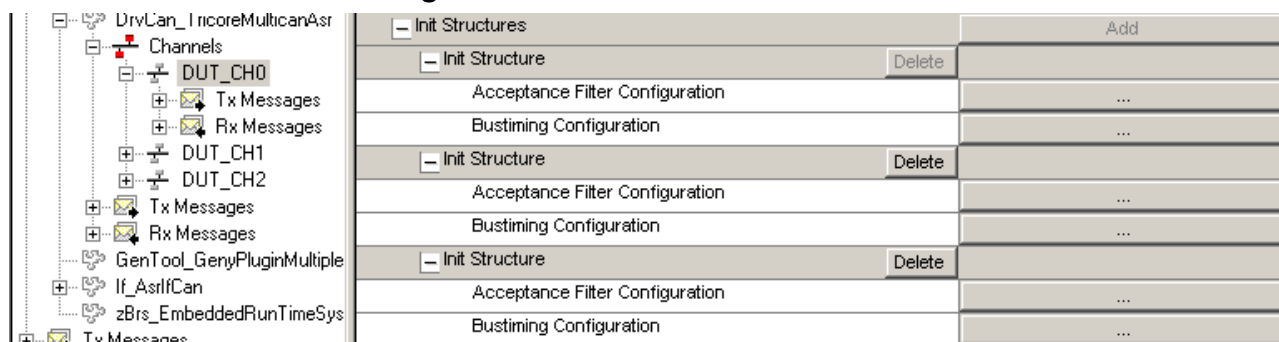


Figure 7-5 Init Structure Dialog

Add Init Structures to setup multiple baud rate and filter settings that can be selected at Initialization phase of the ECU.

MicroSar3 only: This structure can be selected with `Can_InitStruct()` before call `Can_InitController()`.

7.4.5.1 Setup the Filter by pressing the “Acceptance Filter Configuration” Button.

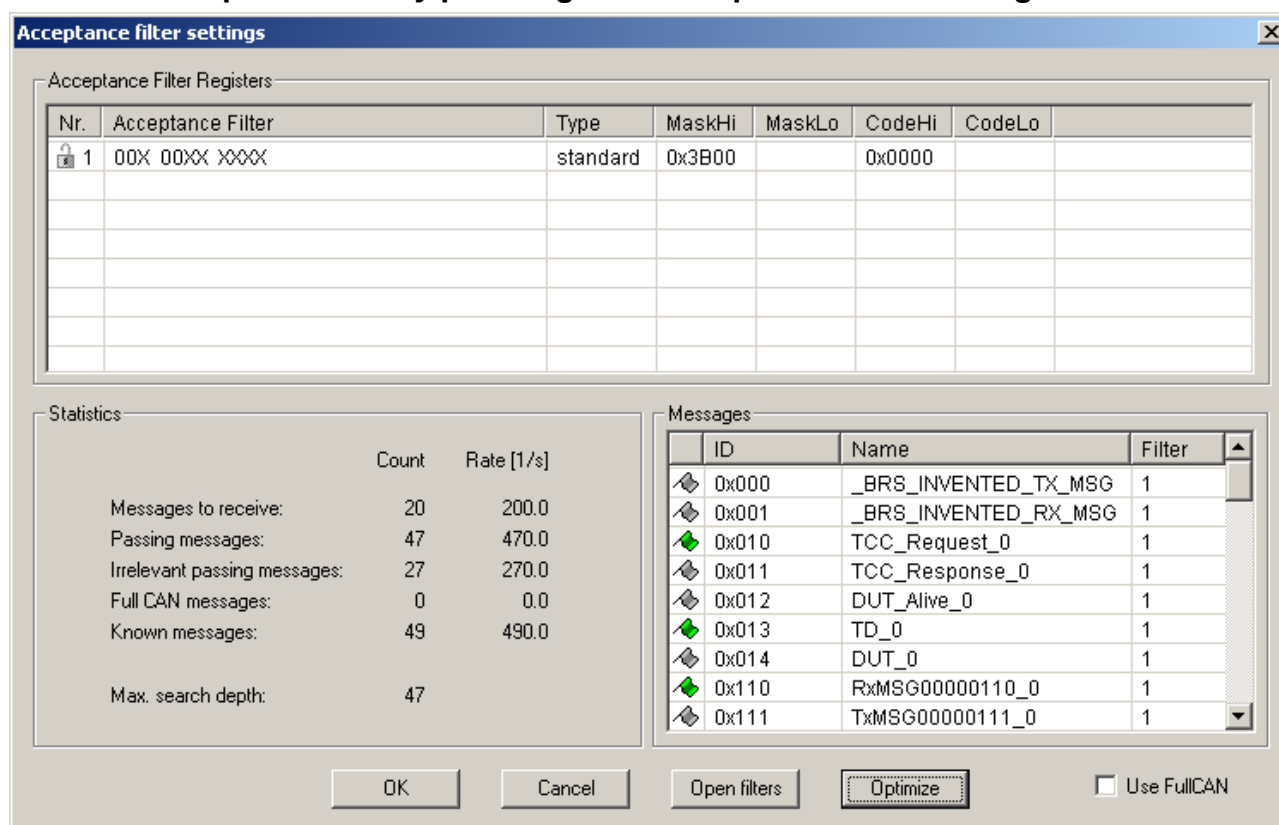
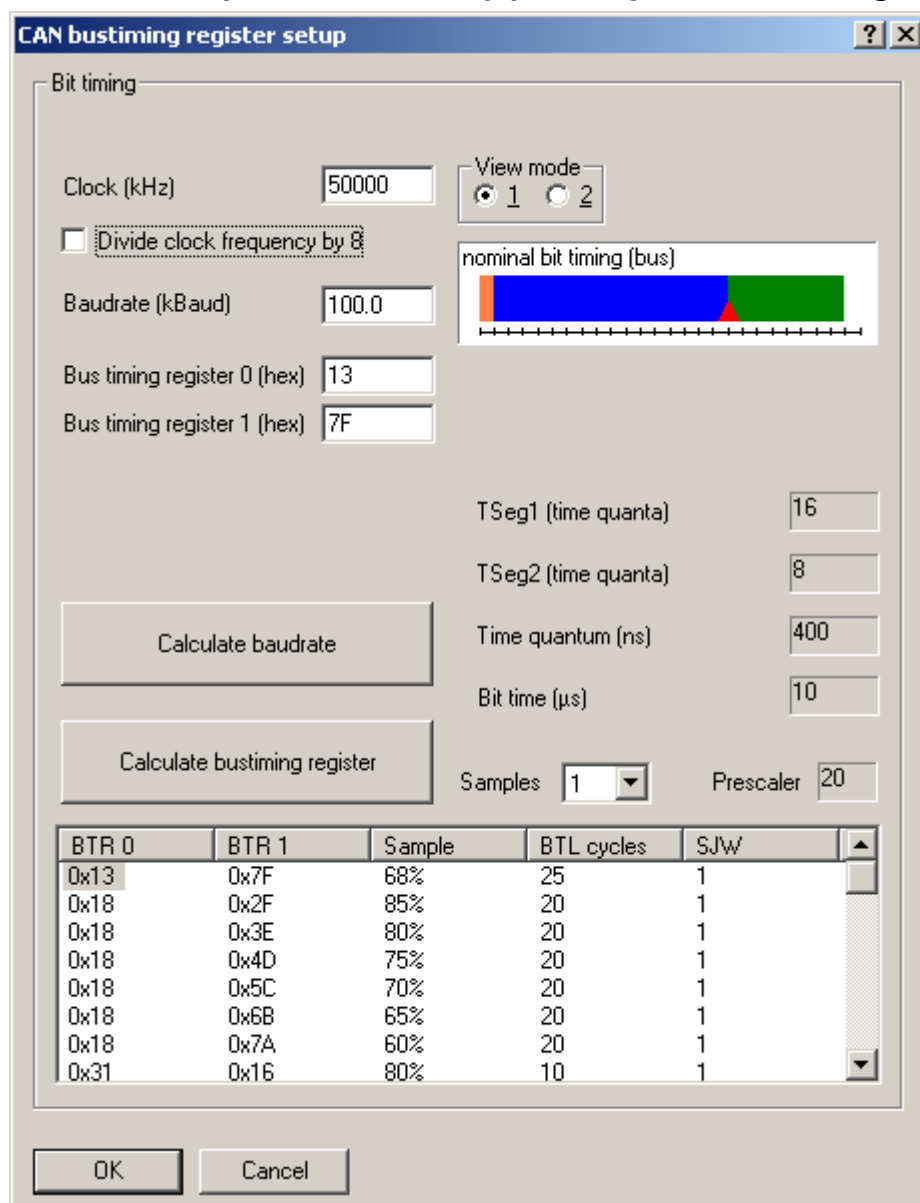


Figure 7-6 Setup Filter Dialog

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values The default value is written in bold	Description
Setup				
Acceptance Filter	Post-build	Integer	Mask, code	Each ID –bit is represented by a “0/1/X”, means must match a “0” or “1” or must not match “X”
Open filters		Action	-	Open the filter complete to receive all IDs
Optimize		Action	-	Open the filter automatic to just receive the IDs in database. “Use FullCAN” try to put as much messages in FullCAN objects to better optimize filters
Messages-Window			Messages that will be received via ranges (like NM messages) will have always a grey flag in view. They will be included in “Irrelevant passing messages” amount.	Open the filter automatic to just receive the IDs in database. “Use FullCAN” try to put as much messages in FullCAN objects to better optimize filters

Table 7-3 Filter Parameter description

7.4.5.2 Setup the Baud rate by pressing the “Bus timing Configuration” Button.



The dialog box is titled "CAN bustiming register setup". It contains the following fields and controls:

- Bit timing** section:
 - Clock (kHz): 50000
 - ☐ Divide clock frequency by 8
 - Baudrate (kBaud): 100.0
 - Bus timing register 0 (hex): 13
 - Bus timing register 1 (hex): 7F
 - View mode: ☒ 1 ☐ 2
 - nominal bit timing (bus): A bar chart showing the timing distribution with orange, blue, and green segments.
- Buttons: "Calculate baudrate" and "Calculate bustiming register".
- Parameters:
 - TSeg1 (time quanta): 16
 - TSeg2 (time quanta): 8
 - Time quantum (ns): 400
 - Bit time (µs): 10
 - Samples: 1 (dropdown)
 - Prescaler: 20
- Table:

BTR 0	BTR 1	Sample	BTL cycles	SJW
0x13	0x7F	68%	25	1
0x18	0x2F	85%	20	1
0x18	0x3E	80%	20	1
0x18	0x4D	75%	20	1
0x18	0x5C	70%	20	1
0x18	0x6B	65%	20	1
0x18	0x7A	60%	20	1
0x31	0x16	80%	10	1
- Buttons: "OK" and "Cancel".

Figure 7-7 Baud Rate Dialog

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
Clock	Post-build	Integer	CAN clock	Set the CAN cell clock.
Divide clock frequency by 8	none	Bool	false	Do not use this feature. (not supported)
Baudrate	Post-build	Integer	Baud rate	Set baud rate to be used for this channel
Calculate		Action	-	It is possible to calculate possible hardware register settings out of baud rate or vice versa.

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
Sample, BTL cycles, SJW	Post-build	Select	Sample Point	Select the sample point and sync phase related to your bus physics

Table 7-4 Baud rate Parameter description

7.5 Configuration with da DaVinci Configurator

See Online help within DaVinci Configurator and BSWMD file for parameter settings.

8 AUTOSAR Standard Compliance

8.1 Limitations / Restrictions

Category	Description	Version
Functional	No support for L-PDU callout (AUTOSAR 3.2.1), but support 'Generic Precopy' instead	3.2.1
Functional	No support for multiple read and write period configuration	3.2.1
API	"Symbolic Name Values" may change their values after precompile phase so do not use it for Linktime or Postbuild variants. It's recommended that higher layer generator use Values (ObjectIDs) from EcuC file. Vector CAN Interface does so.	3.0.6

8.2 Vector Extensions

Refer to Chapter "[Features](#)" listed under "**AUTOSAR extensions**"

9 Glossary and Abbreviations

9.1 Glossary

Term	Description
GENy	Generation tool for CANbedded and MICROSAR components
High End (license)	Product license to support an extended feature set (see Feature table)

Table 9-1 Glossary

9.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution) 3,3x = AUTOSAR version 3 401 = AUTOSAR version 4.0.1 403 = AUTOSAR version 4.0.3 4x = AUTOSAR version 4.x.x
SWS	Software Specification
Common CAN	Connect two physical peripheral channels to one CAN bus (to increase the amount of FullCAN)
Hardware Loop Check	Timeout monitoring for possible endless loops.

Table 9-2 Abbreviations

10 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com